EX NO:1A	STUDY OF COMPUTER NETWORKS
DATE:	

AIM:

To study the basic concept of computer networks and understand how devices communicate through a network.

COMPUTER NETWORKS:

A computer network is a group of computers and other devices connected together to communicate and share data and resources. It allows users to exchange information, share files, printers, applications, and Internet services. Computer networks are classified based on their coverage area. PAN covers a small personal area, LAN connects devices within a limited area such as a home or office, MAN covers a city or large campus, and WAN connects devices over large geographical areas.

Networks use devices such as switches, routers, modems, and network interface cards (NICs) to connect and manage communication between devices. They may use wired connections, such as Ethernet cables, or wireless technologies, such as Wi-Fi. Computer networks provide fast communication, resource sharing, centralized data access, and Internet connectivity. They are widely used in homes, schools, offices, banks, hospitals, and industries.

PROTOTYPE:

A prototype is a simple model or sample of a product or system that is created before developing the final version. It gives us an idea of how the actual system will look and work. A prototype mainly shows the basic design and important features of the system. It does not need to contain all the functions of the final product. It is created to understand the idea clearly and check whether the design is working properly.

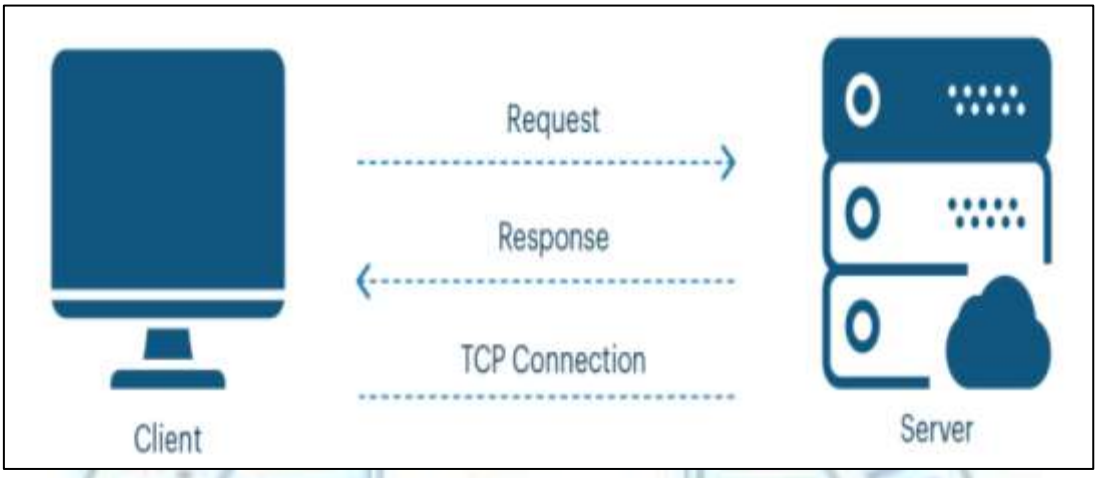
Prototypes can be made using simple drawings, diagrams, wireframes, or interactive designs. They are commonly used in software and application development to show the screens, buttons, menus, and flow of the system. Creating a prototype helps to find mistakes at an early stage. Changes can be made easily before starting the complete development.

TYPES:

1. HTTP (Hyper Text Transfer Protocol)

HTTP Protocol is used to transfer hypertexts over the internet and it is defined by the www(world wide web) for information transfer. It's the protocol that allows web browsers to request web pages from servers and display them to users. However, HTTP is Internet meaning data transmitted over HTTP can be intercepted by malicious actors. It is used to share text, images, and other multimedia files on the

World Wide Web. When a user enters a website URL, the client sends an HTTP request to the server asking for the webpage or specific data. The server then processes this request and sends back an HTTP response containing the required content. This entire exchange happens over a TCP (Transmission Control Protocol) connection, which ensures that the data is sent reliably and in the correct order. HTTP is the communication protocol that makes this client-server interaction possible, allowing users to access web content.

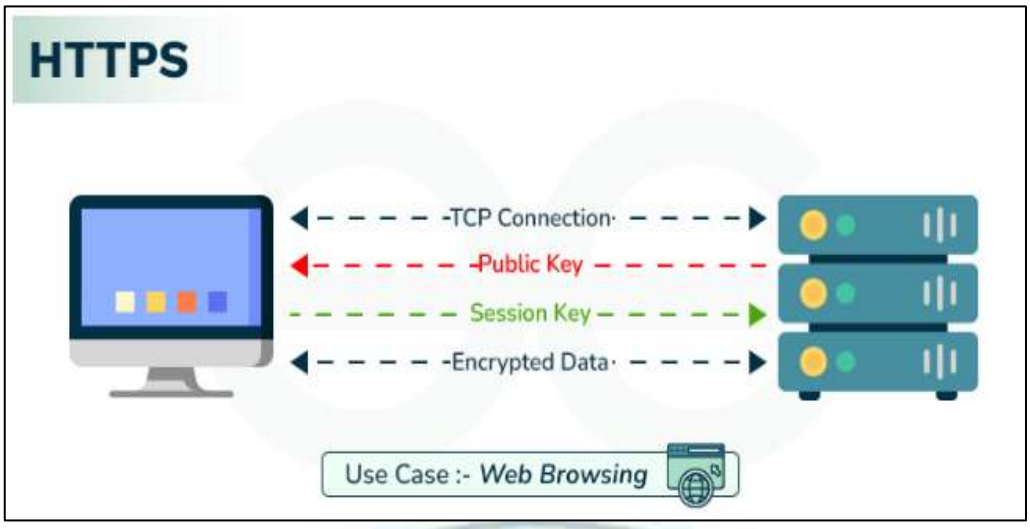


2. HTTPS

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP. It works the same way as HTTP by allowing your browser to request and receive web pages from a server, but with one important difference — it encrypts the data being exchanged. This means any information sent between your browser and the website (like passwords, credit card details, or personal data) is kept private and safe from hackers. HTTPS uses SSL/TLS encryption to protect the connection, making it the preferred and trusted protocol for secure websites, especially for online shopping, banking, or login pages.

The secure communication process in HTTPS involves the following key steps:

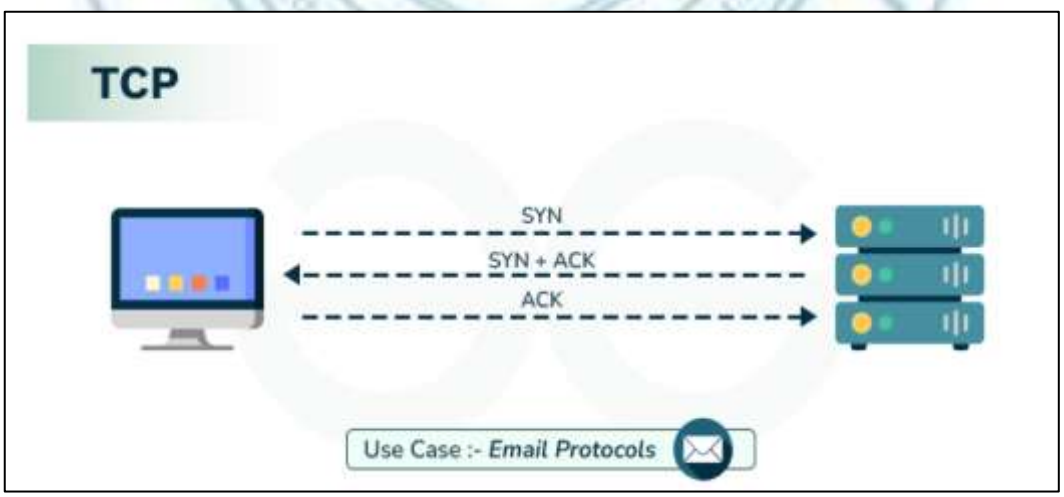
- **TCP Connection**: First, a stable TCP connection is established between the client and the server to start communication.
- **Public Key**: The server sends its public key to the client. This key is part of an SSL certificate and is used to safely exchange information without exposing it to attackers.
- **Session Key**: The client then generates a session key and encrypts it using the server's public key. This ensures that only the server can decrypt and access the session key.
- **Data Encryption**: Once the session key is exchanged, all data transferred between the client and server is encrypted. This keeps personal information, passwords, and other sensitive data safe from hackers.



3. TCP (Transmission Control Protocol)

TCP (Transmission Control Protocol)is a communication protocol that ensures reliable, ordered, and error-checked delivery of data between devices over a network. It breaks data into packets, sends them to the destination, and reassembles them in the correct order. TCP (Transmission Control Protocol) establishes a reliable connection between a client (like your computer) and a **server** using a method called the three-way handshake.

a. SYN (Synchronize): The client starts the process by sending a SYN message to the server, asking to start a connection.
b. SYN + ACK (Acknowledge): The server receives the SYN request and responds with a SYN-ACK message, which means it agrees to the connection and acknowledges the client's request.
c. ACK (Acknowledge): The client replies with an ACK message, confirming the connection.

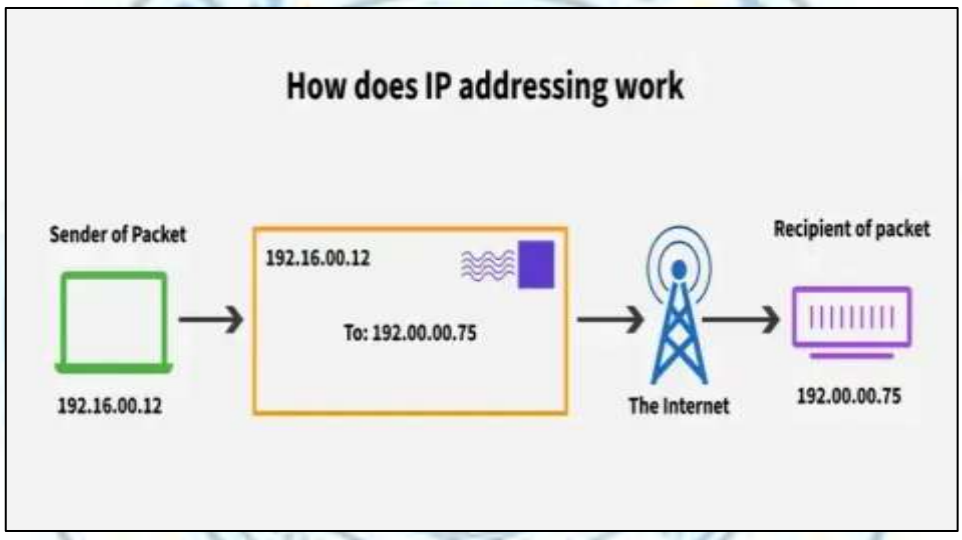


4. IP (Internet Protocol

The Internet Protocol (IP) is the foundational communication protocol that enables devices to send and receive data across the internet and other networks. It acts like a digital postal system, assigning unique

addresses (IP addresses) to devices and ensuring data packets are correctly routed from source to destination.

- The sender device (IP: 192.16.00.12) initiates communication by encapsulating data into a packet, including both its source IP address and the destination IP address.
- The packet is then forwarded to the internet, which serves as the medium responsible for routing the packet based on the destination IP (192.00.00.75).
- Routing protocols and network infrastructure interpret the destination IP and determine the most efficient path for delivering the packet.
- Upon reaching the destination, the recipient device (IP: 192.00.00.75) identifies the packet as intended for it and proceeds to process the received data.
- This process ensures accurate and efficient data delivery between networked devices, forming the core functionality of the Internet Protocol.

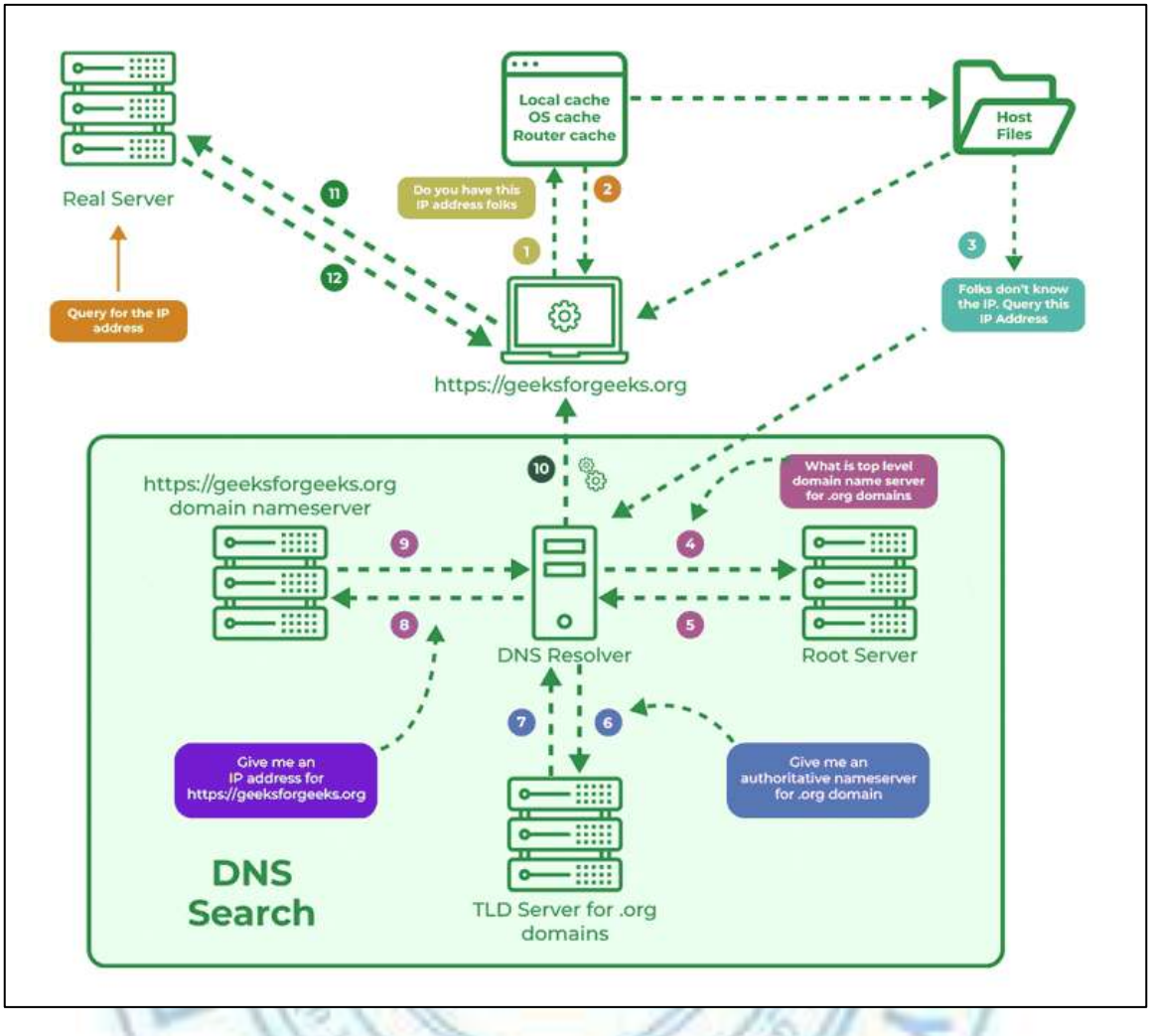


5. DNS (Domain Name System)

DNS (Domain Name System) is like the internet's phonebook. It translates easy-to-remember domain names like www.geeksforgeeks.org into IP addresses like 142.250.77.206, which computers use to identify each other on the network. Without DNS, we'd have to remember long strings of numbers to visit websites. It plays a key role in connecting users to websites quickly and accurately, making browsing the web smooth and user-friendly.

To translate a domain name like geeksforgeeks.org into its corresponding IP address. When a user types a website URL into their browser, the system first checks the local cache, OS cache, or router cache to see if the IP address is already stored. If not found, it moves to the host files, and if still unresolved, the request is sent to a DNS resolver. The resolver contacts a root server, which directs it to the TLD server for geeksforgeeks.org . The TLD server then points to the authoritative nameserver for geeksforgeeks.org, which finally returns the IP address. The resolver provides this IP to the user's system, which then contacts

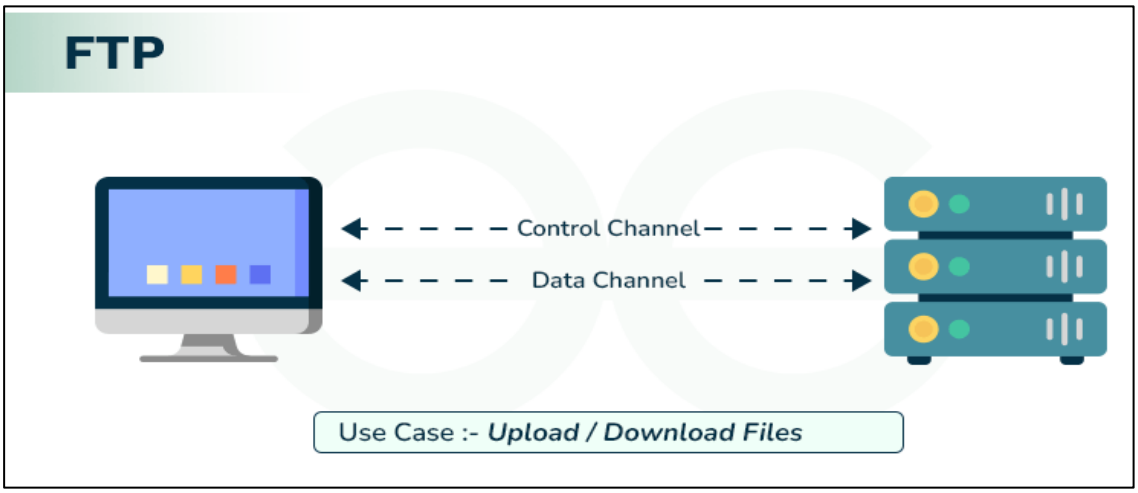
the real server to load the website. This entire process happens in milliseconds, ensuring a smooth browsing experience.



6. FTP (File Transfer Protocol

FTP (File Transfer Protocol)is a standard network protocol used to transfer files between a client and a server over the internet or a local network. It allows users to upload, download, delete, or manage files on a remote server. FTP is commonly used for website management, file sharing, and data backup. However, it does not encrypt data, making it less secure compared to modern alternatives like SFTP or FTPS.

FTP uses two separate connections: the Control Channel and the Data Channel. The Control Channel is used to send commands (like login requests or file operation instructions), while the Data Channel handles the actual transfer of files (uploading or downloading). This separation allows efficient and organized file management over a network. A common use case is uploading website files to a server or downloading backups from it.



NETWORK TOPOLOGY:

Network topology is the way computers and other devices are arranged and connected in a network. It shows how the devices are connected and how they communicate with each other. Network topology helps us understand the structure of a network. It is useful for designing, setting up, and managing a computer network. Different topologies are used depending on the size and requirements of the network.

1. Mesh Topology

Mesh Topology is a network structure in which each device is connected to multiple other devices, creating several communication paths within the network. It provides high redundancy and reliability.

- Devices are interconnected with dedicated links
- Supports multiple paths for data transmission
- Can be full mesh or partial mesh

There are two types of Mesh topologies:

1. Full Mesh Topology:

In a full mesh network, every node is directly connected to all other nodes. If there are N nodes, each node has (N − 1) connections. This provides maximum redundancy and reliability.

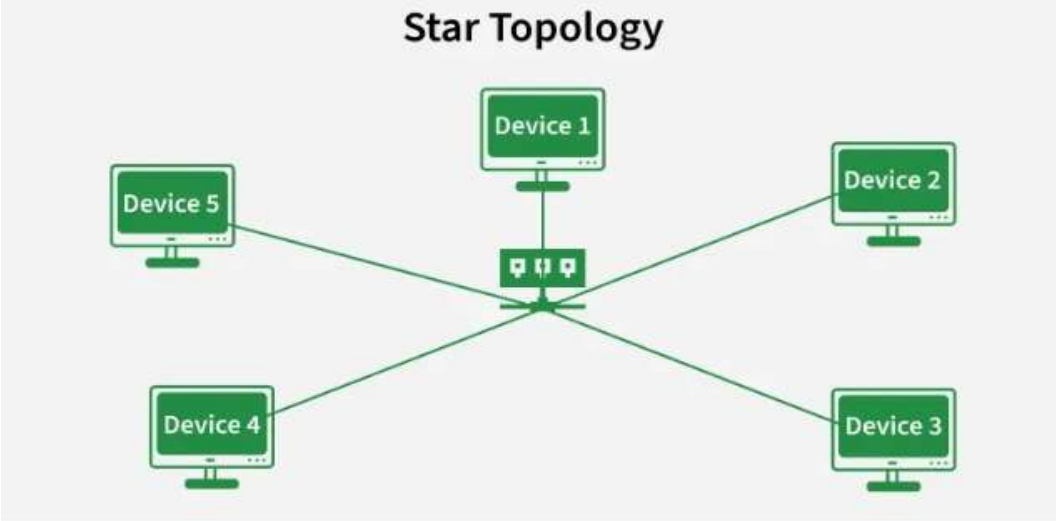
2. Partial Mesh Topology:

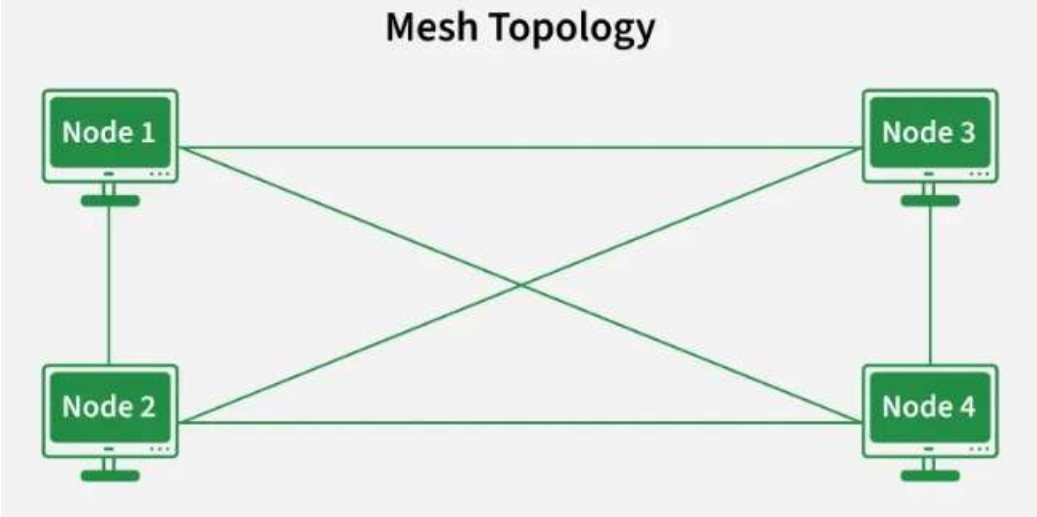
In a partial mesh network, only selected nodes are interconnected. Not every node is directly connected to all others. This approach reduces cost and complexity while still maintaining reasonable reliability.

3. Star Topology

Star Topology is a network layout in which all devices are connected to a central device such as a switch or hub. All communication between devices passes through this central point.

- Each device has a dedicated connection to the central node
- Central device controls data transmission
- Commonly used in Local Area Networks (LANs)

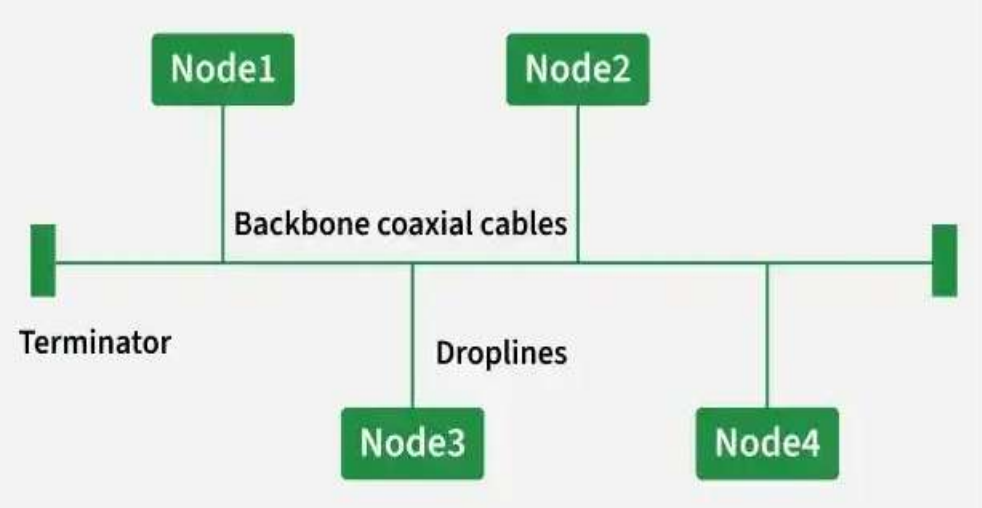




3. Bus Topology

Bus Topology is a network structure in which all devices are connected to a single main communication cable called the backbone. Data sent by any device travels along this common cable and is received by the intended device.

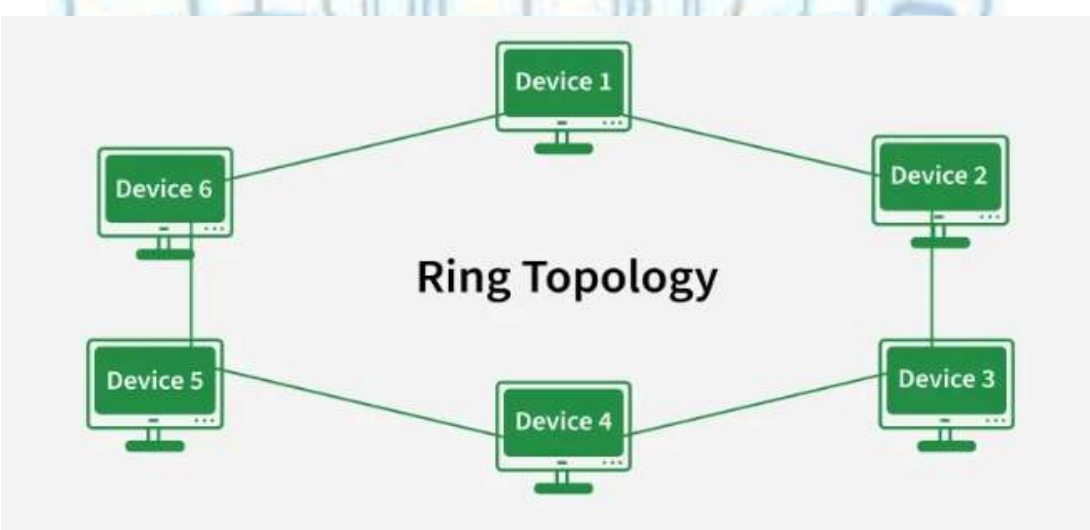
- Uses a single backbone cable
- All devices share the same communication channel
- Terminators are placed at both ends of the cable



4. Ring Topology

Ring Topology is a network configuration in which each device is connected to two other devices, forming a closed circular path. Data travels around the ring in a fixed direction until it reaches its destination.

- Forms a continuous circular structure
- Each device acts as a repeater
- Data passes through intermediate devices

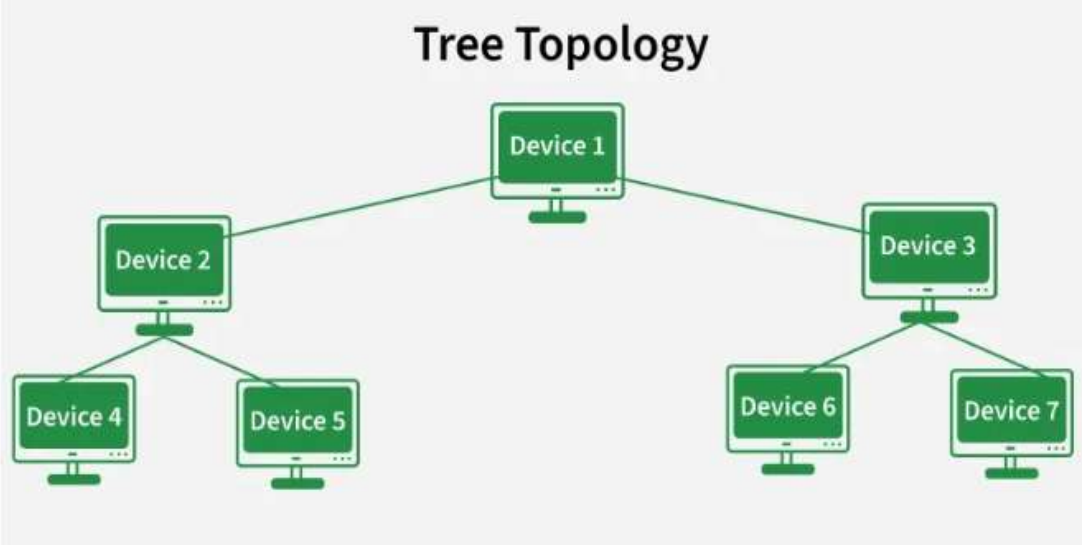


3. Tree Topology

Tree Topology is a hierarchical network structure in which multiple star networks are connected to a central backbone cable. It combines features of both bus and star topologies and organizes devices in a parent–child arrangement.

- Follows a hierarchical structure
- Consists of root node and branch nodes

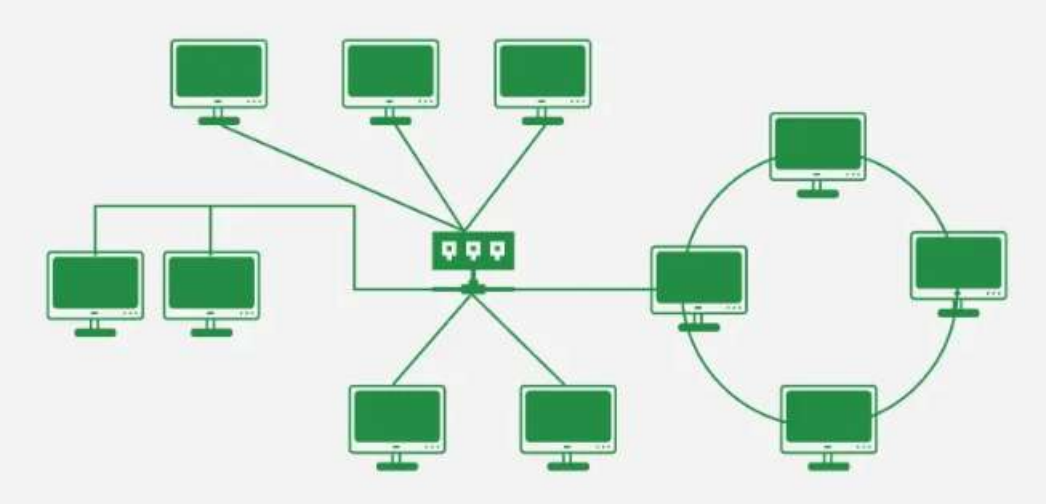
- Uses a backbone cable for connection
- Suitable for large networks



6. Hybrid Topology

Hybrid Topology is a network structure that combines two or more different types of topologies into a single system. It is designed to meet specific organizational requirements by integrating the strengths of multiple network layouts.

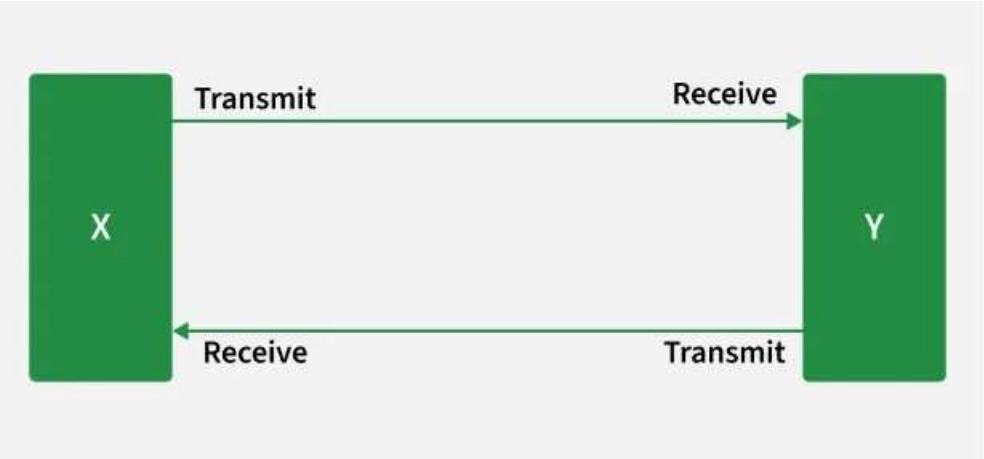
- Formed by combining different topologies
- Customized according to network needs
- Common in large enterprise environments
- Supports complex network structures



7. Point to Point Topology

Point-to-Point Topology is a network configuration in which two devices are directly connected through a dedicated communication link. The connection is exclusive, meaning the entire bandwidth is used only between these two devices.

- Involves only two connected devices
- Uses a single dedicated link
- Data travels directly without intermediaries
- Simple network structure



8. Daisy Chain Topology

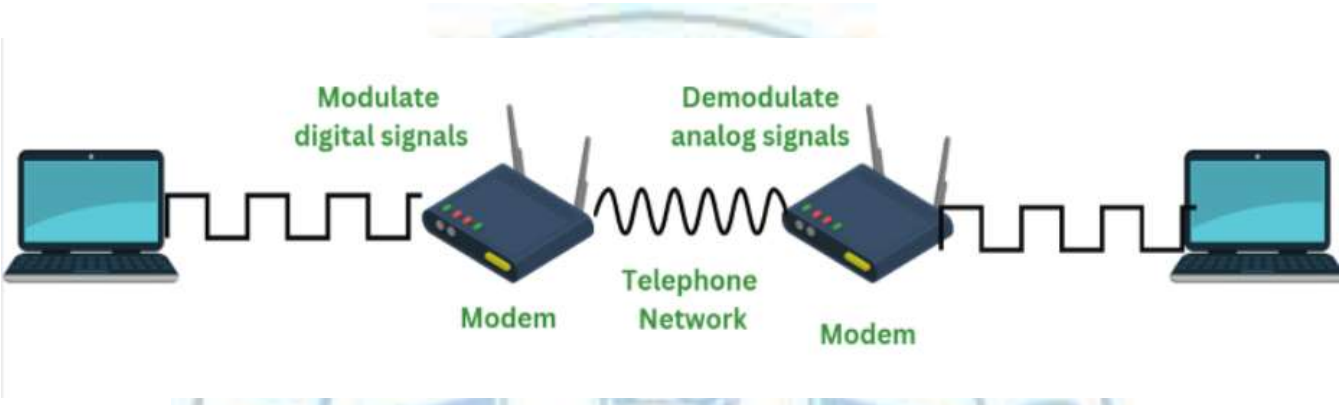
Daisy Chain Topology is a network arrangement in which devices are connected sequentially, one after another, forming a linear chain. Each device is linked to the next device in the sequence.

- Devices are connected in series
- Data passes through intermediate devices
- Simple linear structure
- Often used in small or temporary networks



MODEM:

Modem stands for Modulator/Demodulator. The modem is defined as a networking device that is used to connect devices connected in the network to the internet. The main function of a modem is to convert the analog signals that come from telephone wire into a digital form. In digital form, these converted signals are stored in the form of 0s and 1s. The modem can perform both the task of modulation and demodulation simultaneously. Modems are majorly used to transfer digital data in personal systems. The modem is also known as a signal translator as it translates one signal into another signal by modulating the digital signal into an analog signal for transmission and then demodulates receiving analog signals into digital signals.



TYPES OF MODEM:

1. Optical Modem

In modem, different type of media is used to transfer the signals. Optical Modem is the type of modem that makes use of optical cables instead of using another metallic type of media. The digital data is converted into the pulse of light that is transmitted on the optical fiber used in the optical Modem.

2. Digital Modem

Digital Modem is defined as a type of modem that is used to convert digital data into digital signals. Digital data is in form of 0s and 1s. For this, it performs the process of modulation. Digital Modem modulates the digital data on digital carrier signals for transmission.

3. Cable Modem

Cable modems are defined as a type of modem used to establish a communication between computer systems and the Internet Service Providers. A cable modem helps to access high-speed data through cable TV networks. Such modems are usually connected to desktops or systems and work like external devices.

4. Satellite Modem

Satellite Modems are defined as a type of modem that provides with the internet connection through satellite dishes. This type of modem works by sending the input bits into output radio signals and vice

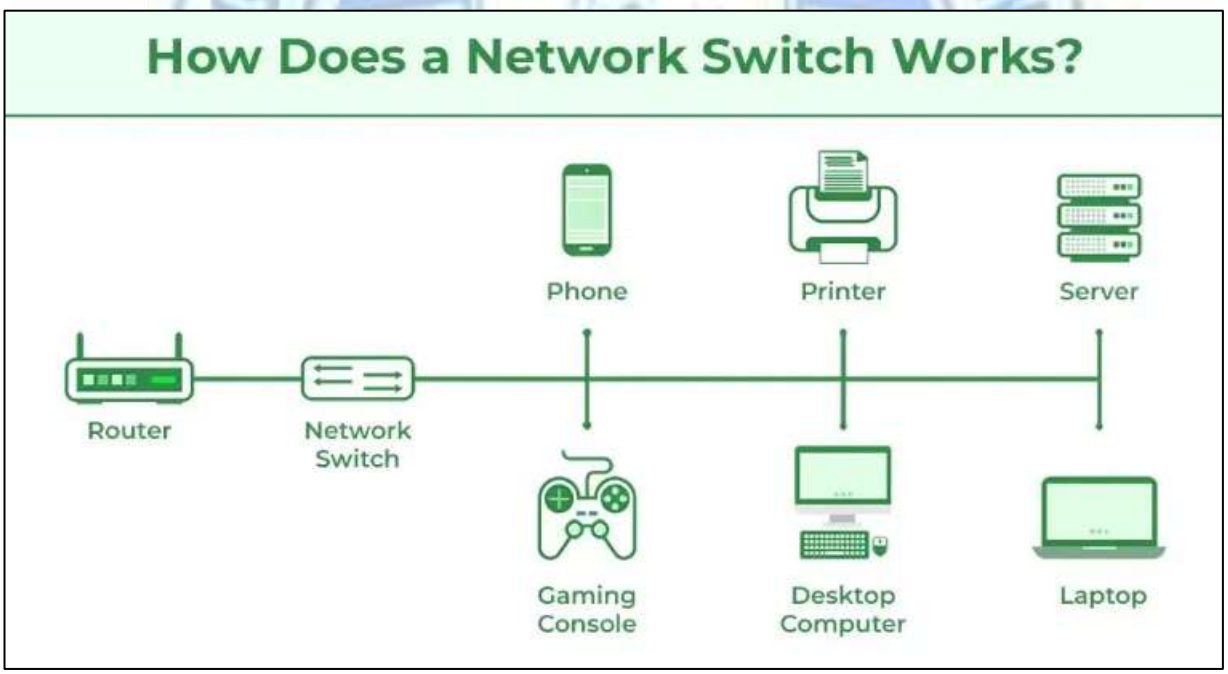
versa. The internet network that is provided by such types of modems is more reliable and efficient as compared to other types of modems.

5. Dial Modem

A Dial Modem is a type of modem that converts data used in telephone and data used on computers. In short dial modem converts between analog form and digital form. The networking devices connected to the computer are all at one end and the telephone line is at another end. This type of modem transmits the data at a speed of 56000 per/sec.

SWITCHING:

Switching is the process of transferring data from one device to another through a network. It helps data reach the correct destination by using network devices such as switches. In a computer network, switching helps connect different devices and manage the flow of data. It allows devices to communicate with each other without sending data to every device in the network. Switching is an important part of computer networks because it helps improve communication, reduce unnecessary traffic, and make data transfer more efficient.



TYPES OF SWITCHING:

1. Message Switching:

Message switching is one of the earliest switching techniques. In this method:

- The entire message is sent to an intermediate node.
- The message is stored temporarily at that node before being forwarded to the next node.
- No dedicated path is established between the sender and receiver.

2. Circuit Switching:

Circuit switching is a connection-oriented method widely used in traditional telephone networks. In this method:

- a dedicated path is established between the sender and receiver before any data transmission begins
- The connection remains reserved for the entire duration of the communication session
- Both ends enjoy a fixed bandwidth and low latency.

3. Packet Switching:

Packet switching is the foundation of modern computer networks, including the Internet. In this method:

- Data is divided into small packets.
- Each packet carries a portion of the message and the destination address.
- Packets may travel through different paths in the network.
- At the destination, packets are reassembled to reconstruct the original message.

RESULT:

Thus, the fundamentals of computer networks were successfully studied, and the basic networking concepts and communication methods were understood.

EX NO: 1B	LEARNING TO USE BASIC COMMANDS AND TRACEROUTE PDUS
DATE:	

AIM:

To learn to use commands like tcpdump, netstat, ifconfig, nslookup and traceroutеping.

COMMANDS:

1. **Tcpdump:**

Display traffic between 2 hosts:

To display all traffic between two hosts (represented by variables host1 and host2):

tcpdump host host1 and host2

Display traffic from a source or destination host only:

To display traffic from only a source (src) or destination (dst) host:

tcpdump src host

tcpdump dst host

Display traffic for a specific protocol

Provide the protocol as an argument to display only traffic for a specific protocol, for example tcp, udp, icmp, arp

tcpdump protocol

For example to display traffic only for the tcp traffic:

tcpdump tcp

Filtering based on source or destination port

To filter based on a source or destination port:

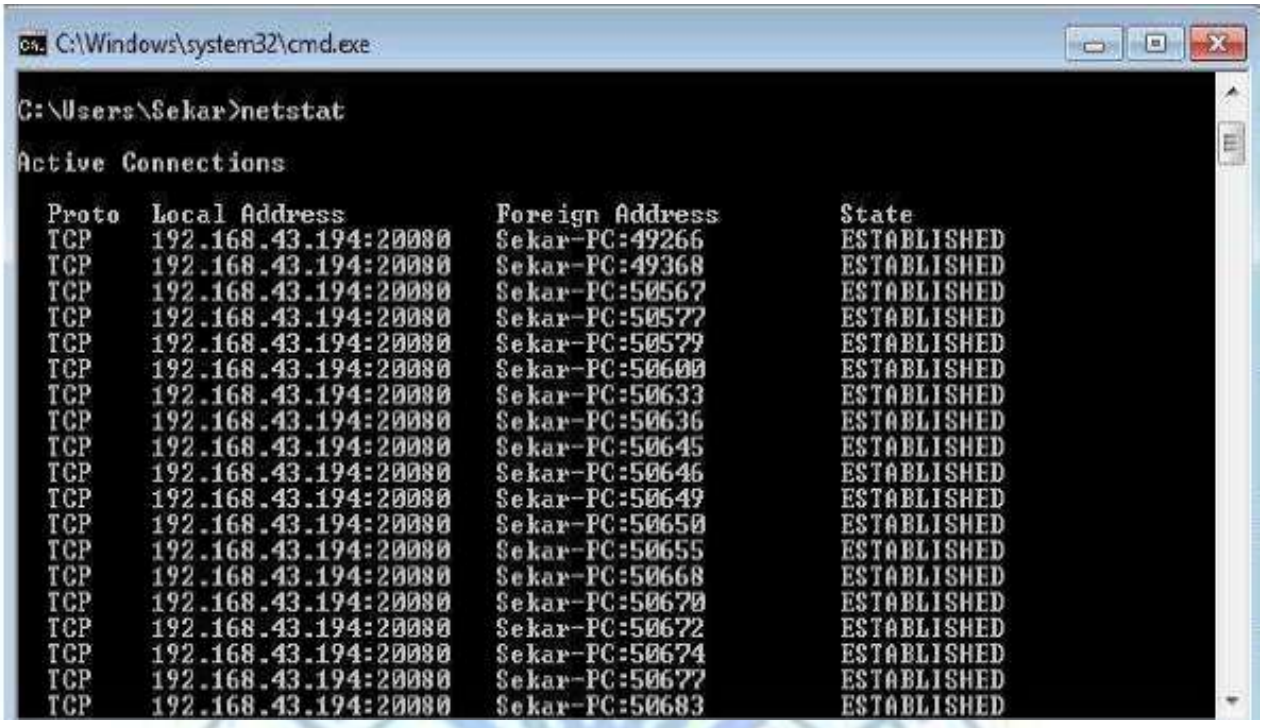
tcpdump src port ftp

tcpdump dst port http

2. **Netstat**

Netstat is a common command line TCP/IP networking available in most versions of Windows, Linux, UNIX and other operating systems. Netstat provides information and statistics about protocols in use and current TCP/IP network connections. The Windows help screen (analogous to a Linux or UNIX for netstat reads as follows: Displays protocol statistics and current TCP/IP network connections.

#netstat



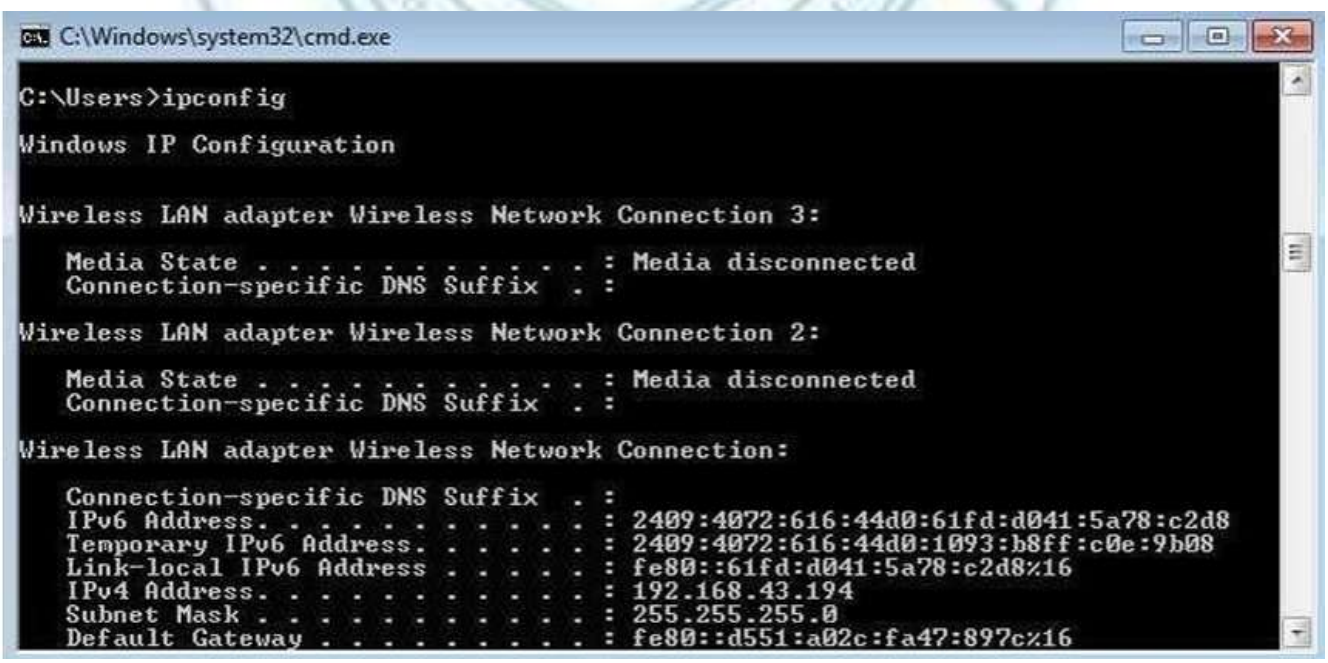
1. ipconfig

In Windows, ipconfig is a console application designed to run from the Windows command prompt.

Using ipconfig

From the command prompt, type ipconfig to run the utility with default options. The output of the default command contains the IP address, network mask, and gateway for all physical and virtual network adapter.

#ipconfig



2. nslookup

The nslookup (which stands for name server lookup) command is a network utility program used to obtain information about internet servers. It finds name server information for domains by querying the Domain Name System. The nslookup command is a powerful tool for diagnosing DNS problems. You know you're experiencing a DNS problem when you can access a resource by specifying its IP address but not its DNS name.

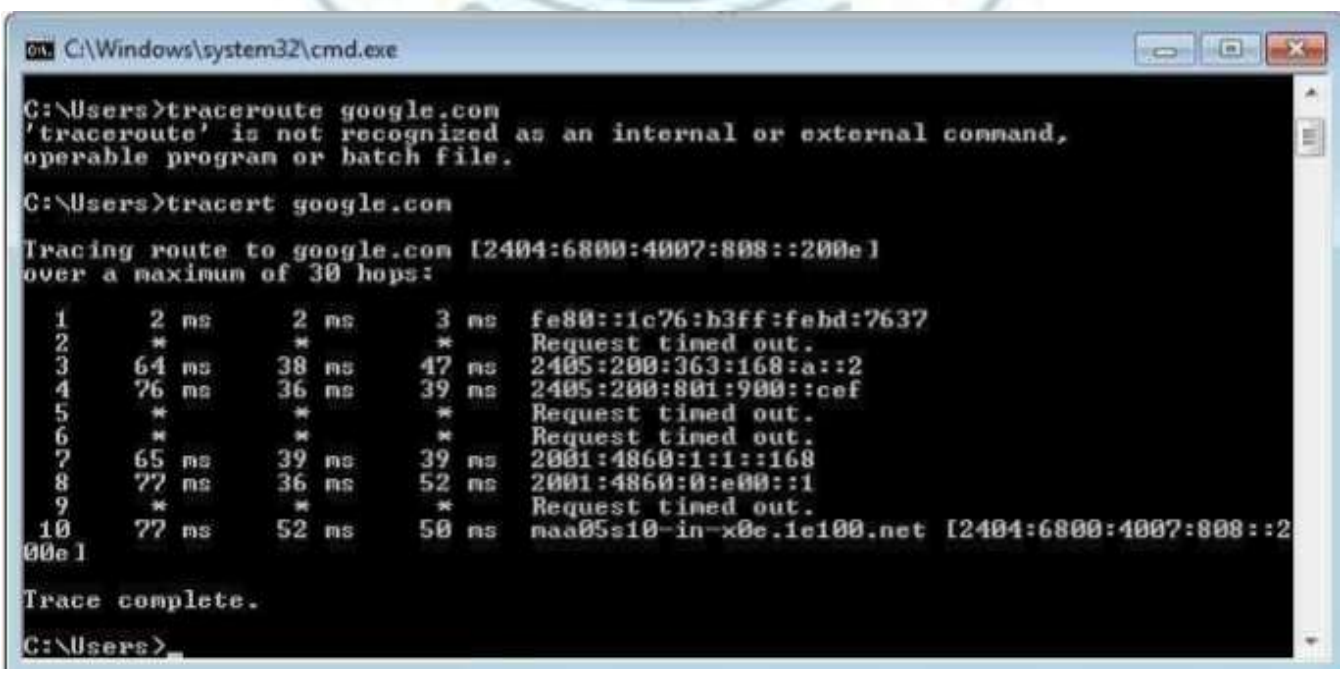
#nslookup

3. Trace route:

Traceroute uses Internet Control Message Protocol (ICMP) echo packets with variable time to live (TTL) values. The response time of each hop is calculated. To guarantee accuracy, each hop is queried multiple times to better measure the response of that particular hop. Traceroute also records the time taken for each hop the packet makes during its route to the destination. Traceroute uses Internet Control Message Protocol (ICMP) echo packets with variable time to live (TTL) values. The response time of each hop is calculated. To guarantee accuracy, each hop is queried multiple times to better measure the response of that particular hop.

Traceroute sends packets with TTL values that gradually increase from packet to packet, starting with TTL value of one. Routers decrement TTL values of packets by one when routing and discard packets whose TTL value has reached zero, returning the ICMP error message ICMP Time Exceeded. For the first set of packets, the first router receives the packet, decrements the TTL value and drops the packet because it then has TTL value zero. Proceeding in this way, traceroute uses the returned ICMP Time Exceeded messages to build a list of routers that packets traverse, until the destination is reached and returns an ICMP Echo Reply message.

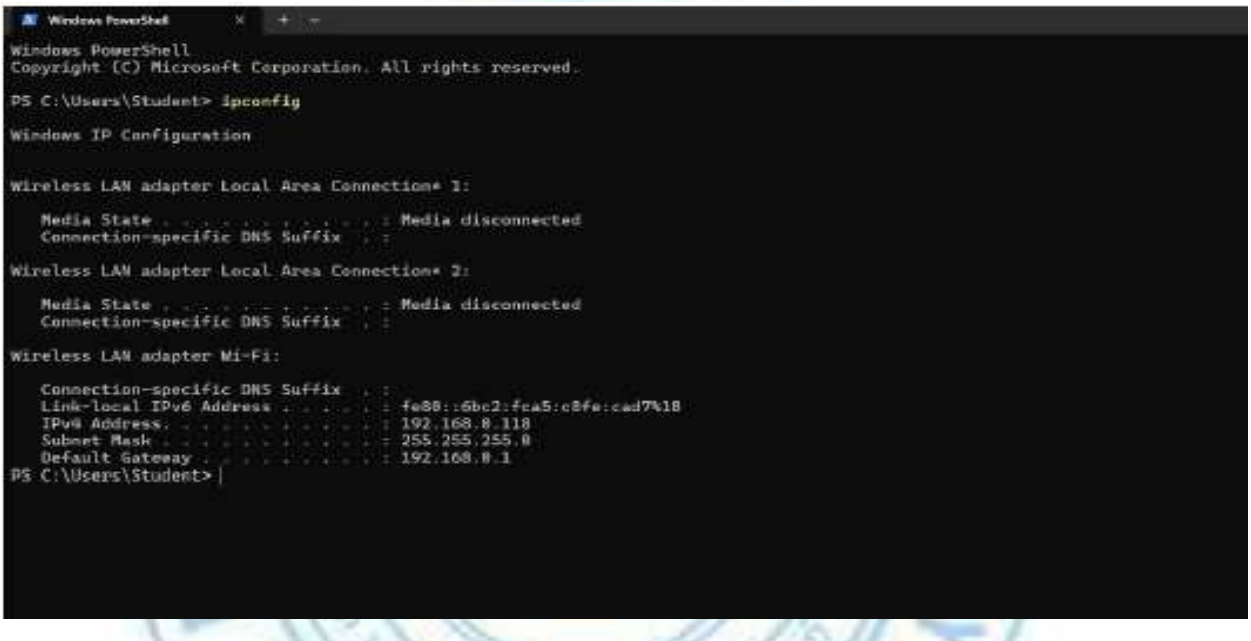
www.google.com.



4. Ping:

The ping command sends an echo request to a host available on the network. Using this command, you can check if your remote host is responding well or not. Tracking and isolating hardware and software problems. Determining the status of the network and various foreign hosts. The ping command is usually used as a simple way to verify that a computer can communicate over the network with another computer or network device. The ping command operates by sending Internet Control Message Protocol (ICMP) Echo Request messages to the destination computer and waiting for a response.

ping172.16.6.2



RESULT:

Thus, the various networks commands like tcpdump, netstat, ifconfig, nslookup and traceroute ping are executed successfully.

EX. NO: 2	DOWNLOAD A WEB PAGE USING TCP SOCKETS
DATE:	

AIM:

To write a java program for socket for HTTP for web page upload and download.

ALGORITHM:

Client:

1. Start the program.
2. Create socket and establish the connection with the server.
3. Read the image to be uploaded from the disk
4. Send the image read to the server
5. Terminate the connection
6. Stop the program.

Server:

1. Start the program
2. Create socket, bind IP address and port number with the created socket and make server a
3. listening server.
4. Accept the connection request from the client
5. Receive the image sent by the client.
6. Display the image.
7. Close the connection.
8. Stop the program

PROGRAM:

Client:

```
Import javax.swing.*;
import java.net.*;
import java.awt.image.*;
import javax.imageio.*;
import java.io.*;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import  javax.imageio.ImageIO;
public class Client
{
```

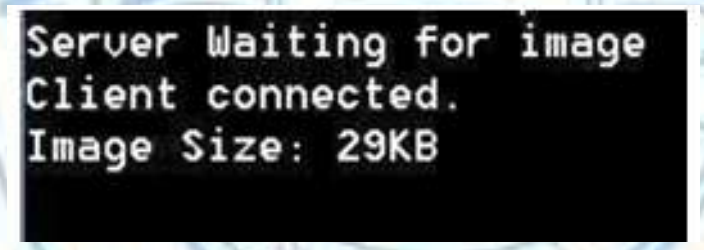
```
public static void main(String args[]) throws Exception
{
Socket soc; BufferedImage img = null;
soc = new Socket("localhost", 4000);
System.out.println("Client is running.");
try{
System.out.println("Reading image from disk. ");
img=ImageIO.read(new File("digital_image_processing.jpg"));
ByteArrayOutputStream baos = new ByteArrayOutputStream();
ImageIO.write(img, "jpg", baos);
baos.flush();
byte[] bytes = baos.toByteArray();
baos.close(); System.out.println("Sending image to server.");
OutputStream out = soc.getOutputStream();
DataOutputStream dos = new DataOutputStream(out);
dos.writeInt(bytes.length);
dos.write(bytes, 0, bytes.length);
System.out.println("Image sent to server. ");
dos.close();
out.close();
}
catch (Exception e)
{
System.out.println("Exception: " + e.getMessage());
soc.close();
}
soc.close();
}
 }
```

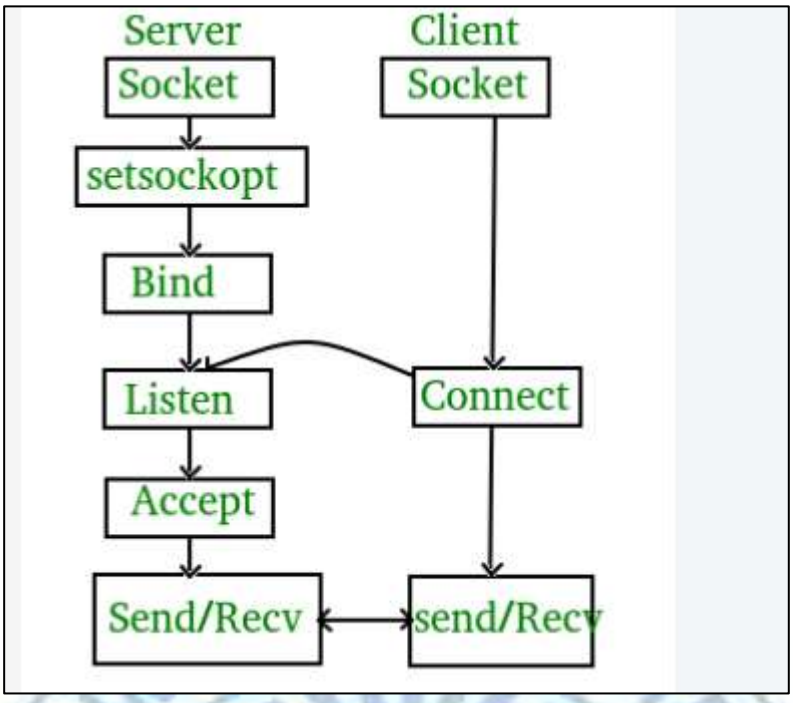
Server:

```
import java.net.*;
import java.io.*;
import java.awt.image.*;
import javax.imageio.*;
import javax.swing.*;
class Server {
public static void main(String args[]) throws Exception
{
 ServerSocket server = null; Socket socket;
server = new ServerSocket(4000);
System.out.println("Server Waiting for image");
socket = server.accept();
System.out.println("Client connected.");
InputStream in = socket.getInputStream();
```

```
DataInputStream dis = new DataInputStream(in);
int len = dis.readInt();
System.out.println("Image Size: " + len / 1024 + "KB");
byte[] data = new byte[len];
dis.readFully(data);
dis.close();
in.close();
InputStream ian = new ByteArrayInputStream(data);
BufferedImage bImage = ImageIO.read(ian);
JFrame f = new JFrame("Server");
ImageIcon icon = new ImageIcon(bImage);
JLabel l = new JLabel();
l.setIcon(ico n);
f.add(l);
f.pack();
f.setVisible(true);
}
}
```

OUTPUT:





RESULT:

Thus, the socket program for HTTP for web page upload and download was developed and executed Successfully.

EX. NO: 3A

DATE:

APPLICATIONS USING TCP SOCKETS: ECHO CLIENT AND ECHO SERVER

AIM:

To write a java program for application using TCP Sockets for Echo client and Echo server.

ALGORITHM:

Client:

Step 1: Start the program.

Step 2: Create the TCP socket.

Step 3: Establish connection with the server.

Step 4: Get the message to be echoed from the user.

Step 5: Send the message to the server.

Step 6: Receive the message echoed by the server.

Step 7: Display the message received from the server.

Step 8: Terminate the connection.

Step 9: Stop the program.

Server:

Step 1: Start the program.

Step 2: Create TCP socket, make it a listening socket.

Step 3: Accept the connection request sent by the client for connection establishment.

Step 4: Receive the message sent by the client.

Step 5: Display the received message.

Step 6: Send the received message to the client from which it receives.

Step 7: Close the connection when client initiates termination and server becomes a listening server, waiting for clients.

Step 8: Stop the program.

PROGRAM:

Client:

```
import java.net.*;
import java.io.*;
public class EClient  {
public static void main(String arg[])  {
Socket c = null;
String line;
```

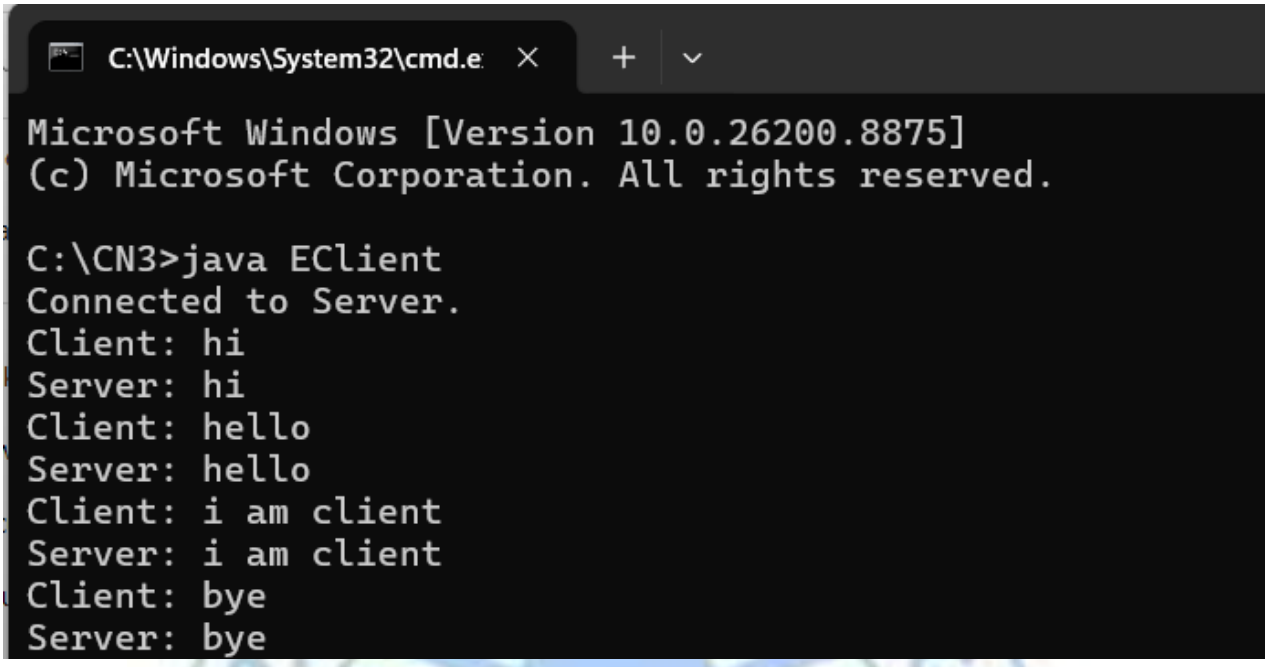
```
DataInputStream is, is1;
PrintStream os;
try
{
InetAddress ia = InetAddress.getLocalHost();
c = new Socket(ia, 9000);
System.out.println("Connected to Server.");
}
catch (IOException e)
{
System.out.println("Connection failed: " + e.getMessage());
 return;
}
try
{
os = new PrintStream(c.getOutputStream());
is = new DataInputStream(System.in);
is1 = new DataInputStream(c.getInputStream());
while (true)  {
System.out.print("Client: ");
line = is.readLine();
os.println(line);
String serverReply = is1.readLine();
System.out.println("Server: " + serverReply);
if (line.equalsIgnoreCase("exit"))  {
System.out.println("Closing connection...");
break;
}
}
c.close();
}
catch (IOException e)  {
System.out.println("Socket Closed or IO Error: " + e.getMessage());
}
}
}
```

Server:

```
import java.net.*;
import java.io.*;
public class eServer  {
public static void main(String args[]){
ServerSocket s = null;
String line;  DataInputStream is;
PrintStream ps;  Socket c = null;
```

```
try
{
s = new ServerSocket(9000);
}
catch (IOException e) {
System.out.println(e);
}
try
{
c = s.accept();
is = new DataInputStream(c.getInputStream());
ps = new PrintStream(c.getOutputStream());
while (true)  {
line = is.readLine();
s.println(line);
}
} catch (IOException e) {
System.out.println(e);
}
}
}
```

OUTPUT:



RESULT:

Thus, the java program for application using TCP Sockets for Echo client and Echo server were executed successfully and their outputs were verified.

EX. NO: 3B	APPLICATIONS USING TCP SOCKETS: CHAT
DATE:	

AIM:

To write a java program for application using TCP Sockets for Chat.

ALGORITHM:

Client:

Step 1: Start the program.

Step 2: Create the UDP datagram socket

Step 3: Get the request message to be sent from the user

Step 4: Send the request message to the server

Step 5: Wait for the reply message from the server

Step 6: Receive the reply message sent by the server

Step 7: Display the reply message received from the server and repeat the steps from 3 to 7.

Step 8: Stop the program.

Server:

Step 1: Start the program.

Step 2: Create UDP datagram socket, make it a listening socket

Step 3: Receive the request message sent by the client

Step 4: Retrieve the client's IP address from the request message received.

Step 5: Display the received message.

Step 6: Get the reply message from the user.

Step 7: Send the reply message to the client and repeat the steps from 3 to 6.

Step 8: Stop the program.

PROGRAM:

Client:

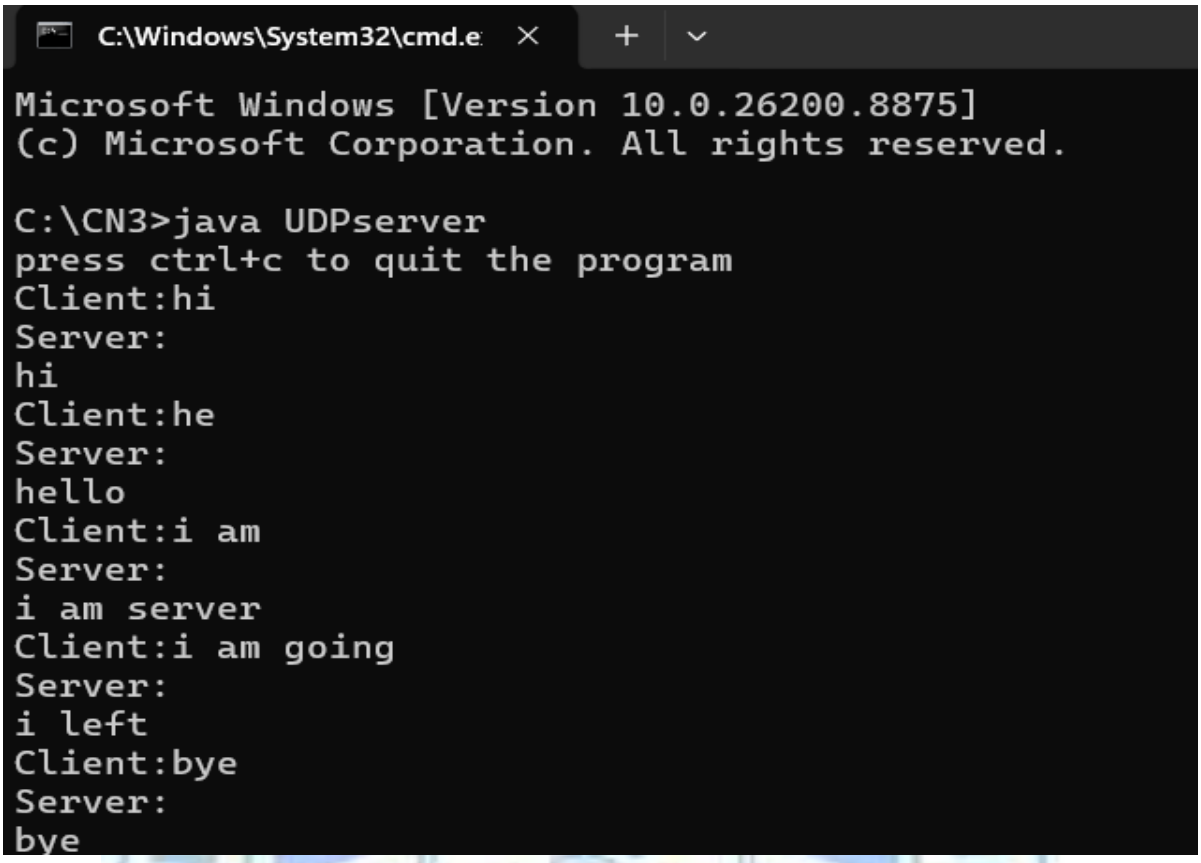
```
import java.io.*;
import java.net.*;
class UDPclient  {
public static DatagramSocket ds;
public static int clientport = 789, serverport = 790;
```

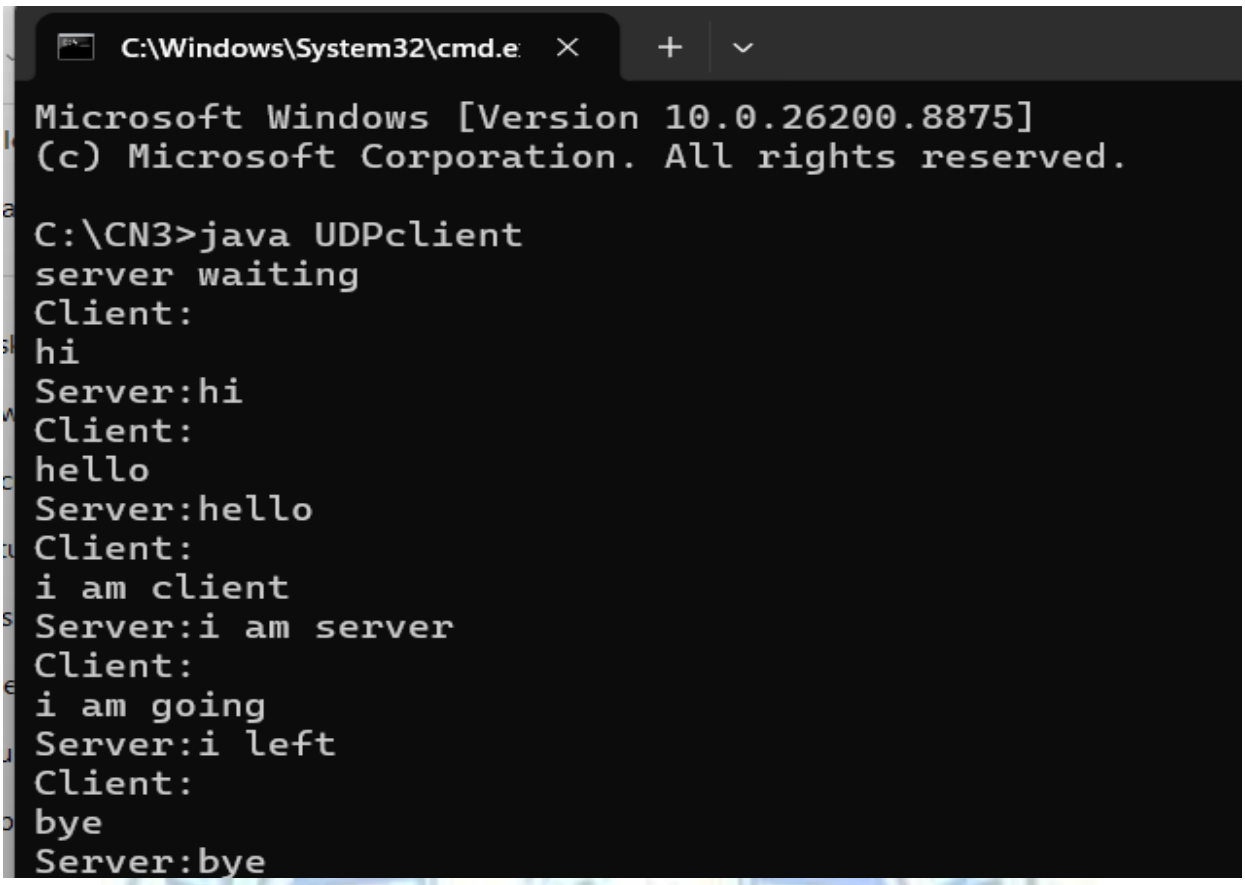
```
public static void main(String args[]) throws Exception  {
byte buffer[] = new byte[1024];
ds = new DatagramSocket(serverport);
BufferedReader dis = new BufferedReader(newInputStreamReader(System.in));
System.out.println("server waiting");
InetAddress ia = InetAddress.getLocalHost();
while (true)  {
System.out.println("Client:");
String str = dis.readLine();
if (str.equals("end"))break;
buffer = str.getBytes();
ds.send(new DatagramPacket(buffer, str.length(), ia, clientport));
DatagramPacket p = new DatagramPacket(buffer, buffer.length);
ds.receive(p);
String psx = new String(p.getData(), 0, p.getLength());
System.out.println("Server:" + psx);
}
}
}
```

Server:

```
import java.io.*;
import java.net.*;
class UDPserver  {
public static DatagramSocket ds;
public static byte buffer[] = new byte[1024];
public static int clientport = 789, serverport = 790;
public static void main(String args[]) throws Exception  {
ds = new DatagramSocket(clientport);
System.out.println("press ctrl+c to quit the program");
BufferedReader dis = new BufferedReader(newInputStreamReader(System.in));
InetAddress ia = InetAddress.getLocalHost();
while (true)  {
DatagramPacket p = new DatagramPacket(buffer, buffer.length);
ds.receive(p);
String psx = new String(p.getData(), 0, p.getLength());
System.out.println("Client:" + psx);
 System.out.println("Server:");
String str = dis.readLine();
if (str.equals("end")) break;
buffer = str.getBytes();
ds.send(new DatagramPacket(buffer, str.length(), ia, serverport));
}
}
}
```

OUTPUT:


C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\CN3>java UDPserver
press ctrl+c to quit the program
Client:hi
Server:
hi
Client:he
Server:
hello
Client:i am
Server:
i am server
Client:i am going
Server:
i left
Client:bye
Server:
bye



RESULT:

Thus, the java program for application using TCP Sockets for chats were executed successfully and their outputs were verified.

EX NO: 3 C	APPLICATIONS USING TCP SOCKETS: FILE TRANSFER
DATE:	

AIM:

To write a java program for application using TCP Sockets for file Transfer.

ALGORITHM:

Client:

Step 1: Create a client socket and connect to the server's IP address and port.

Step 2: Send the desired file name to the server.

Step 3: Receive the file content from the server.

Step 4: Display or save the received file content.

Step 5: Close the connection with the server.

Server:

Step 1: Start the program.

Step 2: Create a server socket and bind it to a specific port.

Step 3: Accept a client connection.

Step 4: Receive the file name from the client.

Step 5: Check if the file exists on the server.

Step 6: If the file exists, read its content.

Step 7: Send the file content to the client.

Step 8: Close the connection and Stop the program.

PROGRAM:

Client:

```
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.net.InetAddress;
import java.net.Socket;
public class FileClient  {
public static void main(String[] args) throws Exception  {
Socket socket = new Socket(InetAddress.getByName("localhost"), 4002);
byte[] contents = new byte[10000];
```

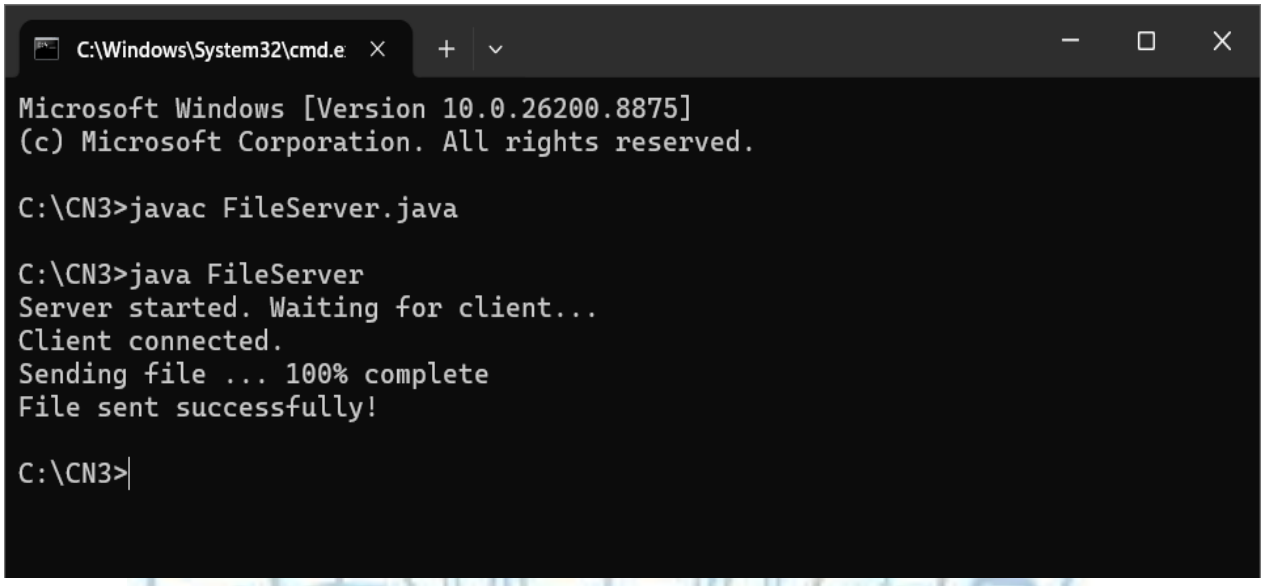
```
FileOutputStream fos = new FileOutputStream ("C:/Users/Admin/Desktop/cn5/received_ text.txt ");
BufferedOutputStream bos = new BufferedOutputStream(fos);
 InputStream is = socket.getInputStream();
int bytesRead = 0;
while ((bytesRead = is.read(contents)) != -1)
{
bos.write(contents, 0, bytesRead); }
bos.flush();
socket.close();
System.out.println("File saved successfully!");
}
}
```

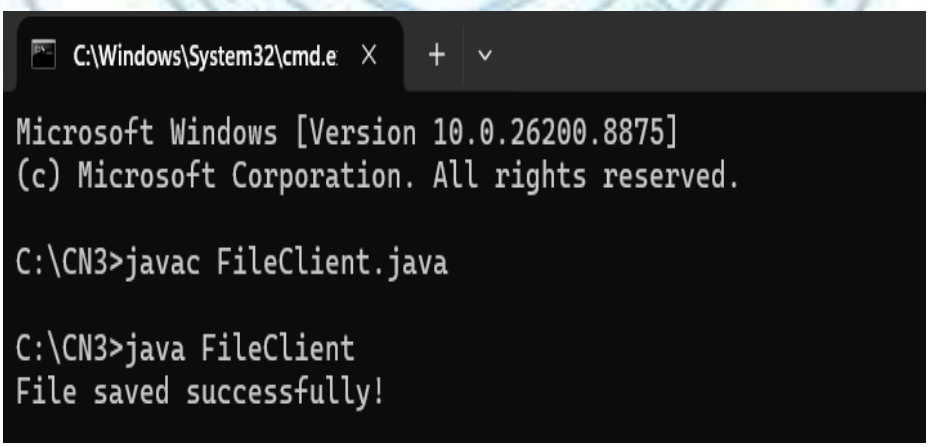
Server:

```
import java.io.BufferedInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.OutputStream;
import java.net.InetAddress;
 import java.net.ServerSocket;
import java.net.Socket;
public class FileServer  {
public static void main(String[] args) throws Exception  {
ServerSocket ssock = new ServerSocket(4002);
Socket socket = ssock.accept();
File file = new File("C:/Users/Admin/Desktop/cn5/text.txt");
FileInputStream fis = new FileInputStream(file);
BufferedInputStream bis = new BufferedInputStream(fis);
OutputStream os = socket.getOutputStream();
byte[] contents; long fileLength = file.length();
long current = 0; long start = System.nanoTime();
while (current != fileLength)
{
int size = 10000;
if (fileLength - current >= size)
{
current += size;
}
   else  {
 size = (int) (fileLength - current);
    current = fileLength;
}
contents = new byte[size];
bis.read(contents, 0, size);
os.write(contents);
System.out.print("Sending file ... " + (current * 100) / fileLength + "%complete!");
```

```
}
os.flush();
socket.close();
ssock.close();
System.out.println("File sent successfully!");
}
}
```

OUTPUT:







RESULT:

Thus, the java program for application using TCP Sockets for file transfer were executed successfully and their outputs were verified.

EX NO: 4	IMPLEMENTATION OF DNS USING UDP SOCKETS
DATE:	

AIM:

To write a java program for implementation of DNS using UDP sockets.

ALGORITHM:

Client:

Step 1: Start the program.
Step 2: Create UDP datagram socket.
Step 3: Get the host name from the client.
Step 4: Send the host name to the server.
Step 5: Wait for the reply from the server.
Step 6: Receive the reply datagram and read the IP address for the requested host name.
Step 7: Display the IP address.
Step 8: Stop the program.

Server:

Step 1: Start the program.
Step 2: Create UDP datagram socket.
Step 3: Create a table that maps host name and IP address.
Step 4: Receive the host name from the client.
Step 5: Retrieve the client‘s IP address from the received datagram
Step 6: Get the IP address mapped for the host name from the table.
Step 7: Display the host name and corresponding IP address.
Step 8: Send the IP address for the requested host name to the client.
Step 9: Stop the program.

PROGRAM:

Client:

```
import java.io.*;
import java.net.*;
public class udpdnsclient  {
public static void main(String args[]) throws IOException  {
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

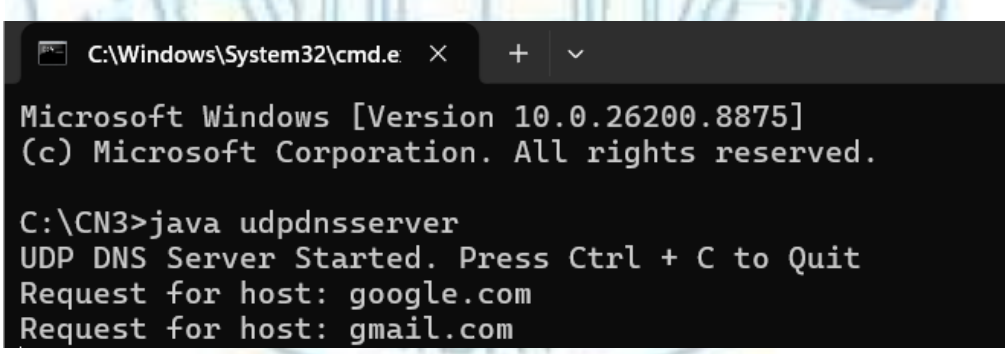
```
DatagramSocket clientsocket = new DatagramSocket();
InetAddress ipaddress;
if (args.length == 0) ipaddress = InetAddress.getLocalHost();
else ipaddress = InetAddress.getByName(args[0]);
byte[] senddata = new byte[1024];
byte[] receivedata = new byte[1024];
 int portaddr = 1362;
System.out.print("Enter the hostname : ");
String sentence = br.readLine();
senddata = sentence.getBytes();
DatagramPacket pack = new DatagramPacket(senddata, senddata.length, ipaddress, portaddr);
clientsocket.send(pack);
DatagramPacket recvpack = new DatagramPacket(receivedata, receivedata.length);
clientsocket.receive(recvpack);
String modified = new String(recvpack.getData(), 0, recvpack.getLength());
System.out.println("IP Address: " + modified); clientsocket.close();
}
}
```

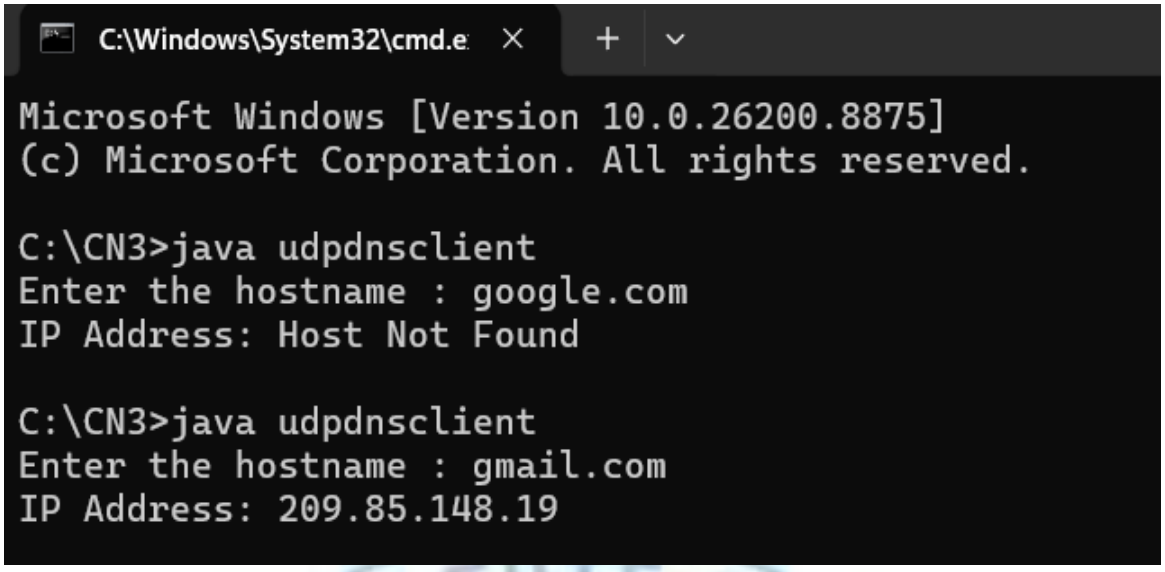
Server:

```
import java.io.*;
import java.net.*;
public class udpdnsserver  {
private static int indexOf(String[] array, String str)  {
str = str.trim();
for (int i = 0; i < array.length; i++)
 {
if (array[i].equalsIgnoreCase(str))
{
return i;
}
}
return -1;
}
public static void main(String[] args) throws IOException  {
String[] hosts = { "yahoo.com", "gmail.com", "cricinfo.com", "facebook.com" };
String[] ips = { "68.180.206.184", "209.85.148.19", "80.168.92.140", "69.63.189.16"};
System.out.println("UDP DNS Server Started. Press Ctrl + C to Quit");
DatagramSocket serverSocket = new DatagramSocket(1362);
while (true)
{
byte[] receiveData = new byte[1024];
byte[] sendData;
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
serverSocket.receive(receivePacket);
```

```
String receivedHost = new String(receivePacket.getData(), 0, receivePacket.getLength()).trim();
System.out.println("Request for host: " + receivedHost);
InetAddress clientAddress = receivePacket.getAddress();
int clientPort = receivePacket.getPort();
String response;
int index = indexOf(hosts, receivedHost);
if (index != -1)
{
response = ips[index];
}
else
{
response = "Host Not Found";
}
 sendData = response.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, clientAddress, clientPort);
serverSocket.send(sendPacket);
}
}
}
```

OUTPUT:





RESULT:

Thus, the java program for implementation of DNS using UDP sockets were executed successfully and their outputs were verified.

EX NO: 5	STUDY OF PACKET CAPTURE AND EXAMINE OF PACKETS USING WIRESHARK
DATE:	

AIM:

To use a tool like Wireshark to capture packets and examine the packets.

ALGORITHM:

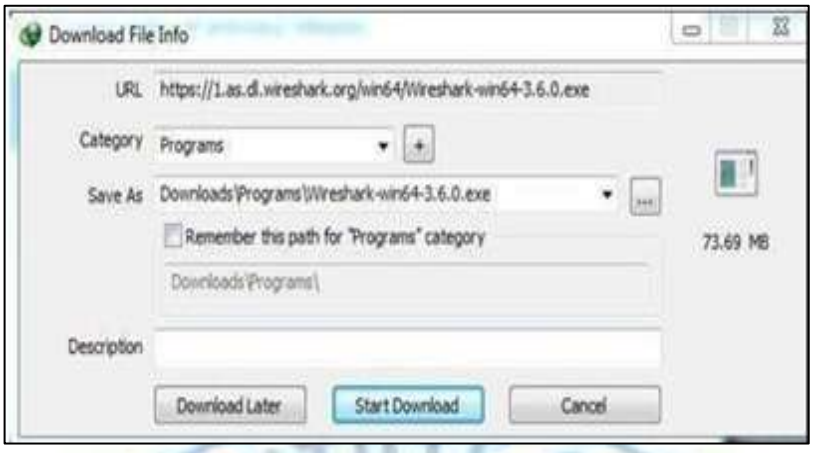
Step 1: Visit the official Wireshark website using any web browser



Step 2: Click on Download, a new webpage will open with different installers of Wireshark.



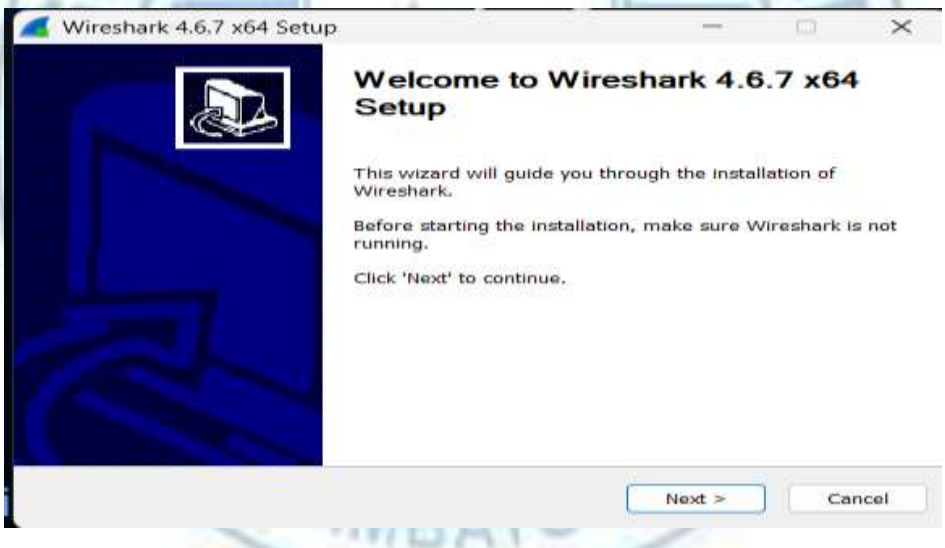
Step 3: Downloading of the executable file will start shortly. It is a small 83.2 MB file that will take some time.



Step 4: Now check for the executable file in downloads in your system and run it.

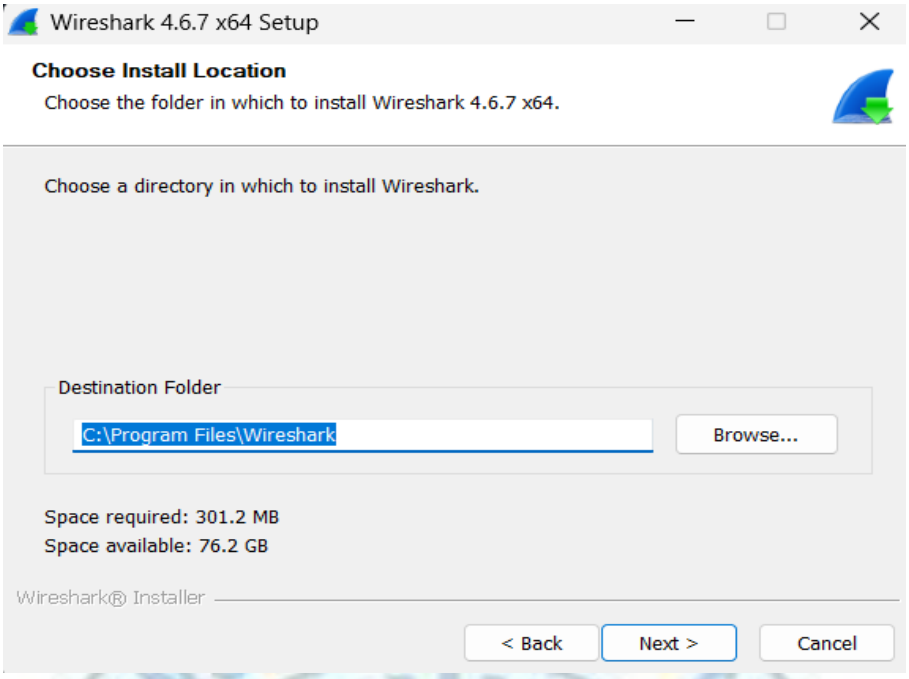
Step 5: It will prompt confirmation to make changes to your system. Click on Yes.

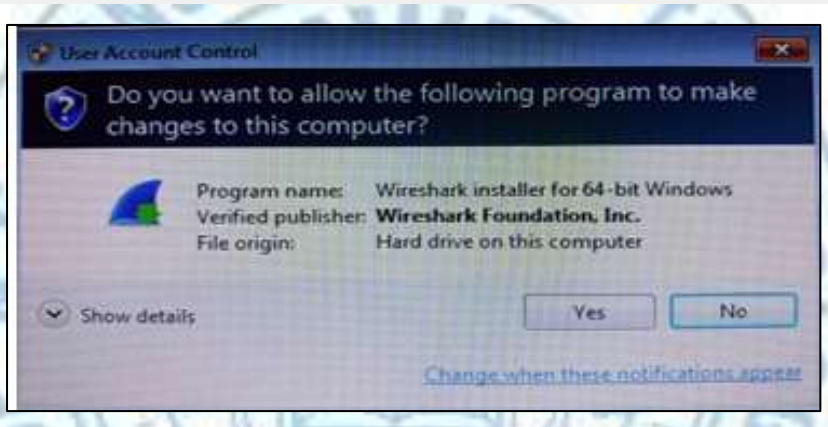
Step 6: Setup screen will appear, click on Next.



Step 7: Next screen has an option to install Npcap which is used with Wireshark to capture packets pcap means packet capture so the install option is already checked don't change anything and click the next button.

Step 8: Next screen is about USB network capturing so it is one's choice to use it or not. Click on install

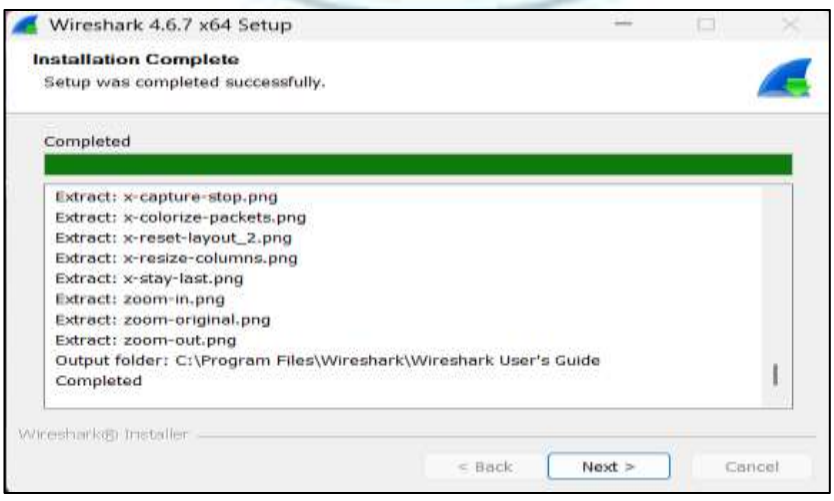




Step 9: After this installation process will start.

Step 10: This installation will prompt for Npcap installation as already checked so the license agreement of Npcap will appear to click on the I Agree button.

Step 11: Next screen is about different installing options of Npcap, don't do anything click on install





Step 12: After this installation process will start which will take only a minute.

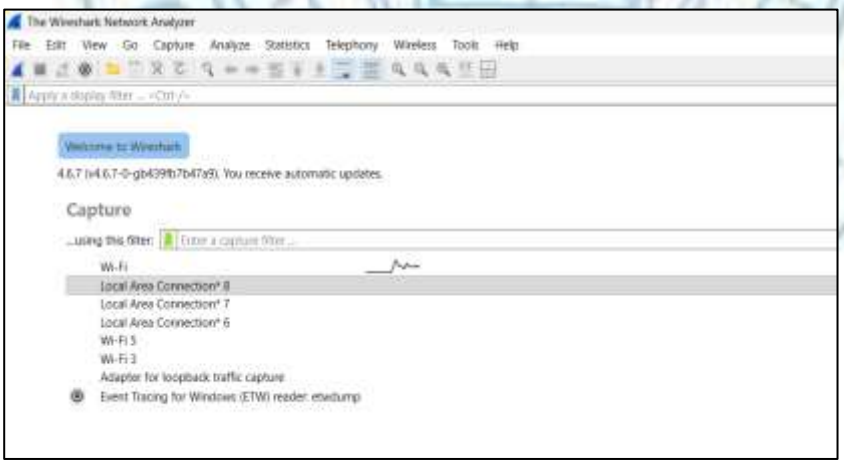
Step 13: After this installation process will complete click on the Next button.

Step 14: Click on Finish after the installation process is complete.

Step 15: After this installation process of Wireshark will complete click on the Next button.

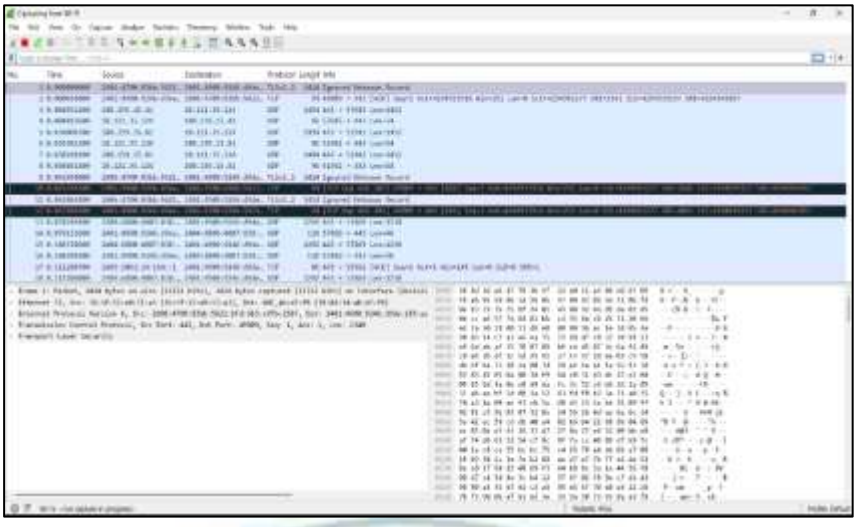
Step 16: Click on Finish after the installation process of Wireshark is complete.

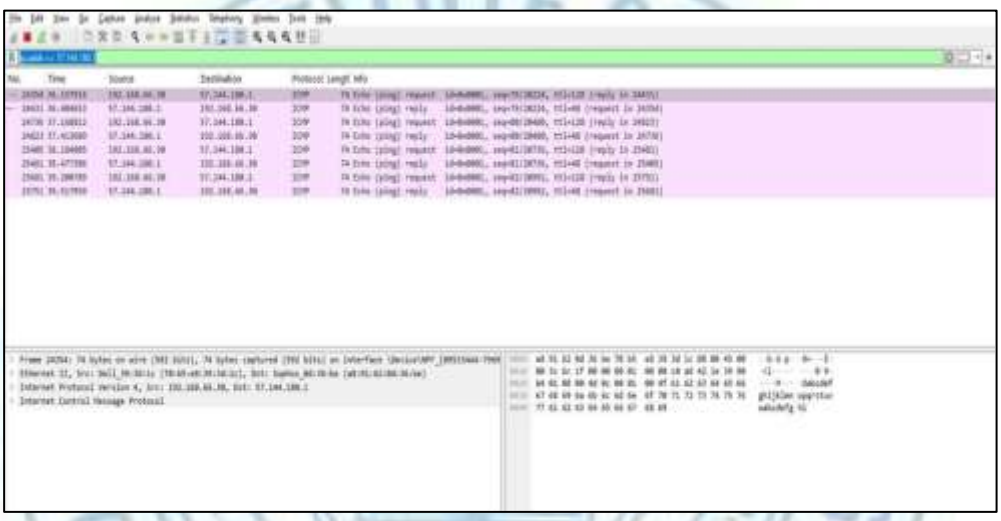
Step 17: Open the Wireshark and choose the capture.



Step 18: Now, you can able to see the Datagrams.

Step 19: Use Ping to get the IP address for any website. For eg: ping google.com





Step 20: Give the IP to the Wireshark to get the datagram transfer.

Ip.addr == 142.250.66.4 is to view all the datagram.

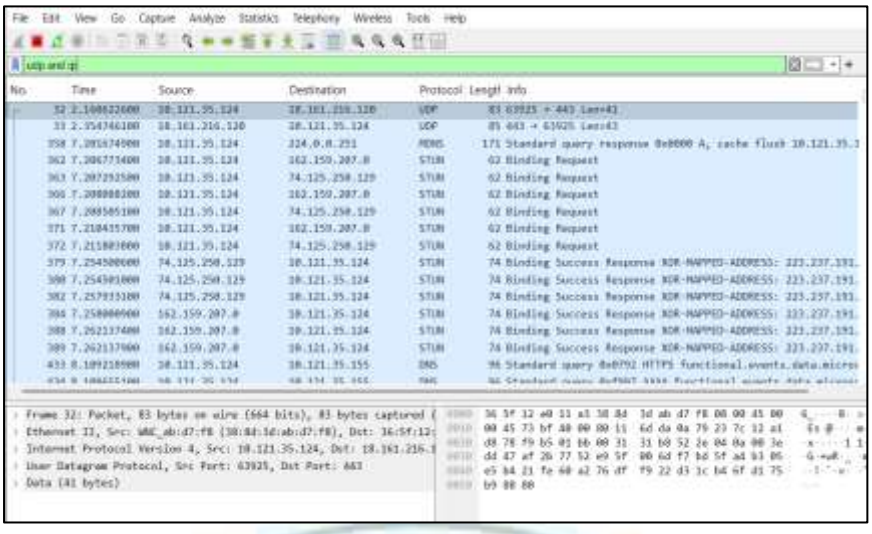
Tcp and Ip.addr == 142.250.66.4 is to view all the TCP datagram. Udp and

Ip.addr == 142.250.66.4 is to view all the UDP datagram

```
C:\Users>ping google.com

Pinging google.com [2404:6800:4007:808::200e] with 32 bytes of data:
Reply from 2404:6800:4007:808::200e: time=877ms
Reply from 2404:6800:4007:808::200e: time=910ms
Reply from 2404:6800:4007:808::200e: time=1011ms
Reply from 2404:6800:4007:808::200e: time=1000ms

Ping statistics for 2404:6800:4007:808::200e:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 877ms, Maximum = 1011ms, Average = 949ms
```



Dr. N.G.P. INSTITUTE OF TECHNOLOGY COIMBATORE

RESULT:

Thus, to use a tool like Wireshark to capture packets and examine the packets were executed successfully and the outputs were verified.

EX NO: 6A	IMPLEMENTATION OF ARP PROTOCOLS
DATE:	

AIM:

To write a java program for simulating ARP and RARP protocols using TCP.

ALGORITHM:

Client:

1. Start the program
2. Create socket and establish connection with the server.
3. Get the IP address to be converted into MAC address from the user.
4. Send this IP address to server.
5. Receive the MAC address for the IP address from the server.
6. Display the received MAC address
7. Terminate the connection

Server:

1. Start the program
2. Create the socket, bind the socket created with IP address and port number and make it a listening socket.
3. Accept the connection request when it is requested by the client.
4. Server maintains the table in which IP and corresponding MAC addresses are stored.
5. Receive the IP address sent by the client.
6. Retrieve the corresponding MAC address for the IP address and send it to the client.
7. Close the connection with the client and now the server becomes a listening server waiting for the connection request from other clients
8. Stop

PROGRAM:

Client:

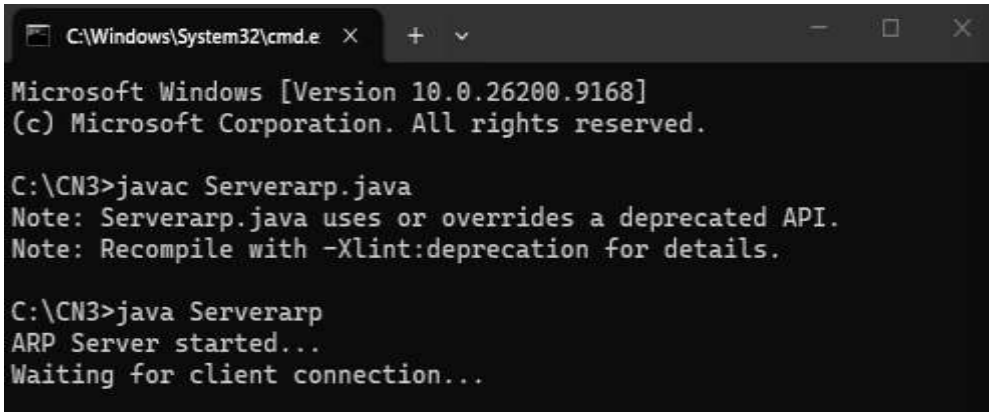
```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientarp {
public static void main(String args[]) {
try {
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
Socket clsct = new Socket("127.0.0.1", 139);
```

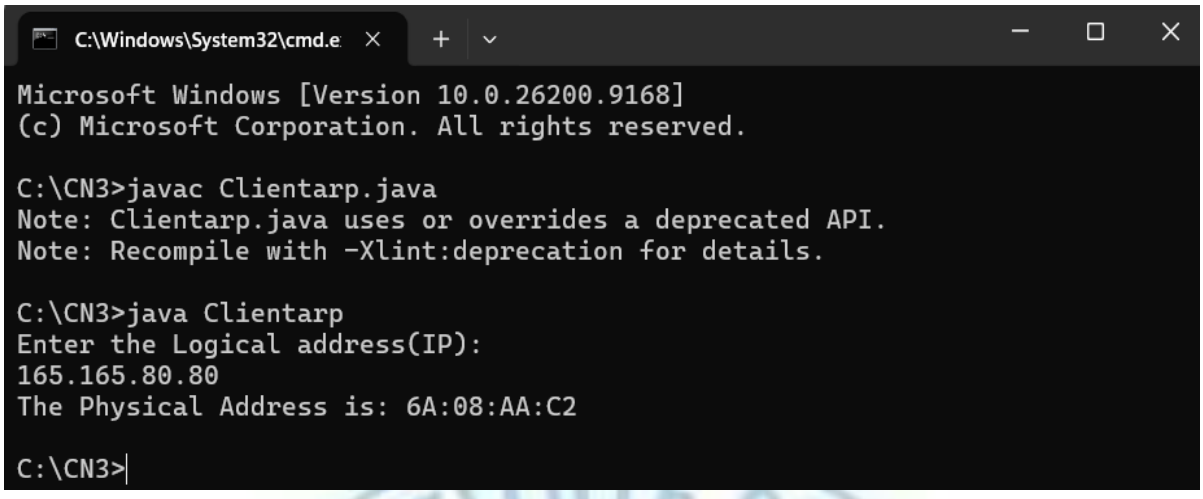
```
DataInputStream din = new DataInputStream(clsct.getInputStream());
DataOutputStream dout = new DataOutputStream(clsct.getOutputStream());
System.out.println("Enter the Logical address(IP):");
String str1 = in.readLine();
dout.writeBytes(str1 + '\n');
String str = din.readLine();
System.out.println("The Physical Address is: " + str); clsct.close();
} catch (Exception e)
{ System.out.println(e);
} } }
```

Server:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverarp {
public static void main(String args[]) {
try {
ServerSocket obj = new ServerSocket(139);
Socket obj1 = obj.accept();
while (true) {
DataInputStream din = new DataInputStream(obj1.getInputStream());
DataOutputStream dout = new DataOutputStream(obj1.getOutputStream());
String str = din.readLine();
String ip[] = { "165.165.80.80", "165.165.79.1" };
String mac[] = { "6A:08:AA:C2", "8A:BC:E3:FA" };
for (int i = 0; i < ip.length; i++)
{ if (str.equals(ip[i])) { dout.writeBytes(mac[i] + '\n');
break; }
} obj.close(); }
} catch (Exception e)
{  System.out.println(e);
} }}}
```

OUTPUT:





RESULT:

Thus, the program for implementing to display simulating ARP protocols was executed successfully and output is verified.

EX. NO:6 B	IMPLEMENTATION OF RARP PROTOCOLS
DATE:	

AIM:

To write a Program for Reverse Address Resolution Protocol (RARP) using UDP

ALGORITHM:

Client:

1. Start the program
2. Create datagram socket
3. Get the MAC address to be converted into IP address from the user.
4. Send this MAC address to server using UDP datagram.
5. Receive the datagram from the server and display the corresponding IP address.
6. Stop

Server:

1. Start the program.
2. Server maintains the table in which IP and corresponding MAC addresses are stored.
3. Create the datagram socket
4. Receive the datagram sent by the client and read the MAC address sent.
5. Retrieve the IP address for the received MAC address from the table.
6. Display the corresponding IP address.
7. Stop

PROGRAM:

Client:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientrarp12 {
public static void main(String args[]) {
try
{
DatagramSocket client = new DatagramSocket();
InetAddress addr = InetAddress.getByName("127.0.0.1");
byte[] sendbyte = new byte[1024];

byte[] receivebyte = new byte[1024];
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter the Physical address (MAC):");
String str = in.readLine();
```

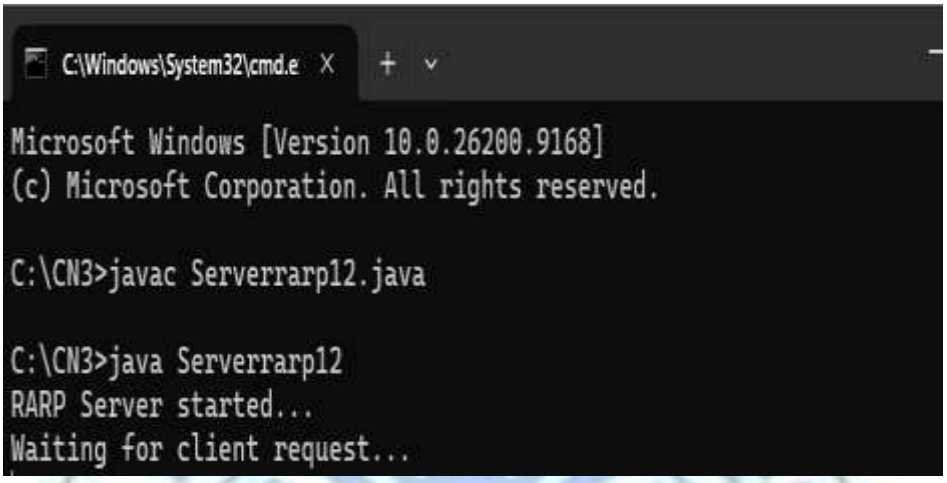
```
sendbyte = str.getBytes();
DatagramPacket sender = new DatagramPacket(sendbyte, sendbyte.length, addr, 1309);
client.send(sender);
DatagramPacket receiver = new DatagramPacket(receivebyte, receivebyte.length);
client.receive(receiver);
String s = new String(receiver.getData());
System.out.println("The Logical Address is(IP): " + s.trim());
client.close();
}
catch (Exception e)
{
System.out.println(e );
}}}
```

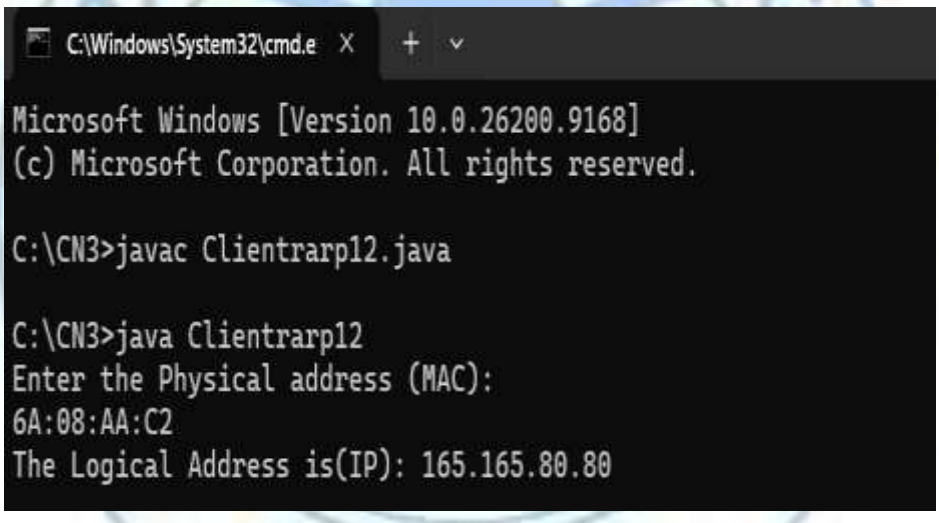
Server:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Serverrarp12 {
public static void main(String args[]) {
try {
DatagramSocket server = new DatagramSocket(1309);
while (true) {
byte[] sendbyte = new byte[1024];
byte[] receivebyte = new byte[1024];
DatagramPacket receiver = new DatagramPacket(receivebyte,receivebyte.length);
server.receive(receiver);
String str = new String(receiver.getData());
String s = str.trim();
InetAddress addr = receiver.getAddress();
int port = receiver.getPort();
String ip[] = { "165.165.80.80", "165.165.79.1" };
String mac[] = { "6A:08:AA:C2", "8A:BC:E3:FA" };
for (int i = 0; i < ip.length; i++)
{
if (s.equals(mac[i]))
{
sendbyte = ip[i].getBytes();
DatagramPacket sender = new DatagramPacke t(sendbyte, sendbyte.length, addr, port);
erver.send(sender);
break;
}
}
break;
}
} catch (Exception e)
```

```
{ System.out.println(e);
} }
}
```

OUTPUT:





RESULT:

Thus, the program for implementing to display simulating RARP protocols was executed successfully and output is verified.

EX. NO: 7	IMPLEMENTATION OF CONGESTION CONTROL ALGORITHMS USING NS.
DATE:	

AIM:

To implement congestion control algorithms using NS2.

PROCEDURE:

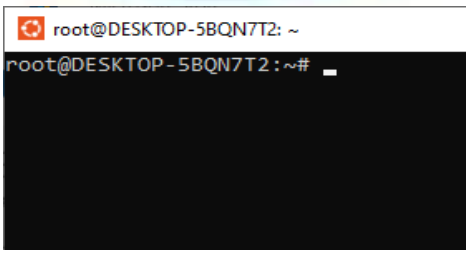
Step1: Download Ubuntu from Microsoft store.



Step2: Download and install Xming from chrome.



Step3: Open Ubuntu.



Step4: Run the following commands in terminal.

i) sudo apt update

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ sudo apt update
[sudo] password for administrator:
Get:1 https://hax4us.github.io/TermuxBlack termuxblack InRelease [5,692 B]
Err:1 https://hax4us.github.io/TermuxBlack termuxblack InRelease
  The following signatures were invalid: EXPKEYSIG 4BEE22F9F994D6C4 lokesh pandey <lkpandey950@gmail.com>
Hit:2 http://security.ubuntu.com/ubuntu focal-security InRelease
Get:3 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 DEP-11 Metadata [212 B]
Get:4 http://security.ubuntu.com/ubuntu focal-security/restricted DEP-11 48x48 Icons [29 B]
Get:5 http://security.ubuntu.com/ubuntu focal-security/restricted DEP-11 64x64 Icons [29 B]
Get:6 http://security.ubuntu.com/ubuntu focal-security/restricted DEP-11 64x64@2 Icons [29 B]
```

ii) sudo apt-get install ns2

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ sudo apt-get install ns2
Reading package lists... Done
Building dependency tree
Reading state information... Done
ns2 is already the newest version (2.35+dfsg-3build1).
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 331 not upgraded.
```

iii) sudo apt-get install nam

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ sudo apt-get install nam
Reading package lists... Done
Building dependency tree
Reading state information... Done
nam is already the newest version (1.15-5build1).
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 331 not upgraded.
```

vi) sudo apt-get install gedit

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ sudo apt-get install gedit
Reading package lists... Done
Building dependency tree
Reading state information... Done
gedit is already the newest version (3.36.2-0ubuntu1).
gedit set to manually installed.
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 331 not upgraded.
```

v) sudo apt-get install tcl

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ sudo apt install tcl
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  tcl8.6
Suggested packages:
  tcl-tclreadline
The following NEW packages will be installed:
  tcl tcl8.6
0 upgraded, 2 newly installed, 0 to remove and 331 not upgraded.
Need to get 19.9 kB of archives.
After this operation, 72.7 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Step 5: Execute gedit filename.tcl and type your code and click save.

```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ gedit exp7.tcl
```



Step6: Run the file by executing the code ns filename.tcl.

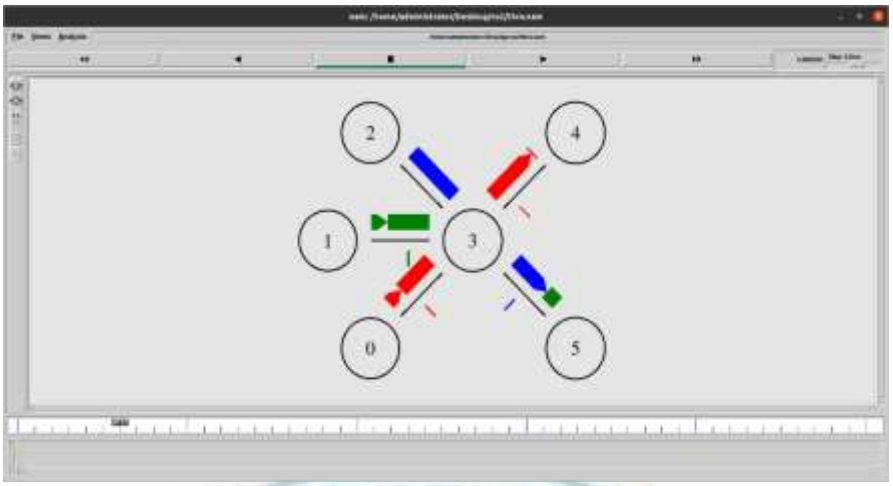
```
administrator@administrator-OptiPlex-3070:~/Desktop/ns2$ ns exp7.tcl
 When configured, ns found the right version of tclsh in /usr/bin/tclsh8.6
but it doesn't seem to be there anymore, so ns will fall back on running the
uild ns if this is a problem.
Simulation is starting...
running nam...
```

PROGRAM:

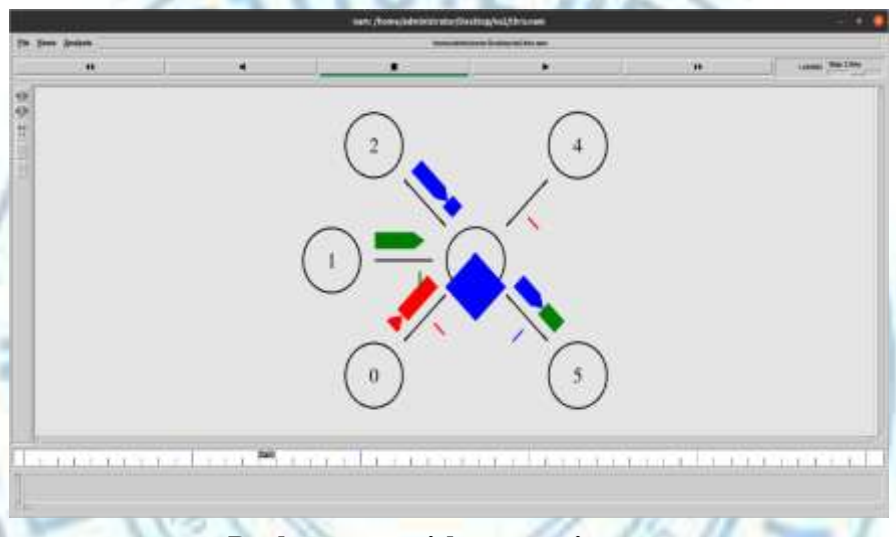
```
set ns [new Simulator]
set nr [open thro_red.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
proc finish {} {
   global ns nr nf
   $ns flush-trace
   close $nf
   close $nr
   exec nam thro.nam &
   exit 0
}
set n0 [$ns node]  ;
set n1 [$ns node]  ;
set n2 [$ns node]  ;
set n3 [$ns node]  ;
set n4 [$ns node]  ;
set n5 [$ns node]  ;
$ns duplex-link $n0 $n3 1Mb 10ms RED
$ns duplex-link $n1 $n3 1Mb 10ms RED
$ns duplex-link $n2 $n3 1Mb 10ms RED
$ns duplex-link $n3 $n4 1Mb 10ms RED
$ns duplex-link $n3 $n5 1Mb 10ms RED
$ns duplex-link-op $n0 $n3 orient right-up
```

```
$ns duplex-link-op $n1 $n3 orient right
$ns duplex-link-op $n2 $n3 orient right-down
$ns duplex-link-op $n3 $n4 orient right-up
$ns duplex-link-op $n3 $n5 orient right-down
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n4 $sink0
$ns connect $tcp0 $sink0
set tcp1 [new Agent/TCP]
$ns attach-agent $n1 $tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n5 $sink1
$ns connect $tcp1 $sink1
set tcp2 [new Agent/TCP]
$ns attach-agent $n2 $tcp2
set sink2 [new Agent/TCPSink]
$ns attach-agent $n5 $sink2
$ns connect $tcp2 $sink2
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $tcp0
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $tcp1
set cbr2 [new Application/Traffic/CBR]
$cbr2 set packetSize_ 500
$cbr2 set interval_ 0.005
$cbr2 attach-agent $tcp2
$tcp0 set fid_ 1
$tcp1 set fid_ 2
$tcp2 set fid_ 3
$ns color 1 Red
$ns color 2 Green
$ns color 3 Blue
$ns at 0.1 "$cbr0 start"
$ns at 0.2 "$cbr1 start"
$ns at 0.3 "$cbr2 start"
$ns at 4.0 "$cbr0 stop"
$ns at 4.2 "$cbr1 stop"
$ns at 4.5 "$cbr2 stop"
$ns at 5.0 "finish"
$ns run
```

OUTPUT:



Packets sent without congestion



Packets sent with congestion

RESULT:

Thus, the congestion control algorithm is implemented in NS2 and verified successfully.

EX NO. 8	STUDY OF TCP/UDP PERFORMANCE USING NS2
DATE:	

AIM:

To study of TCP/UDP performance using NS2

TCP ALGORITHM:

Step 1. Initialize the Simulator

Step 2. Create Nodes

Step 3. Set up Network Links

Step 4. Link Orientations for Visualization

Step 5. Create TCP Traffic Generation Agents

Step 6. Configure Flow Identifiers and Colors

Step 7. Set Traffic Start and Stop Times

Step 8. Finish Procedure

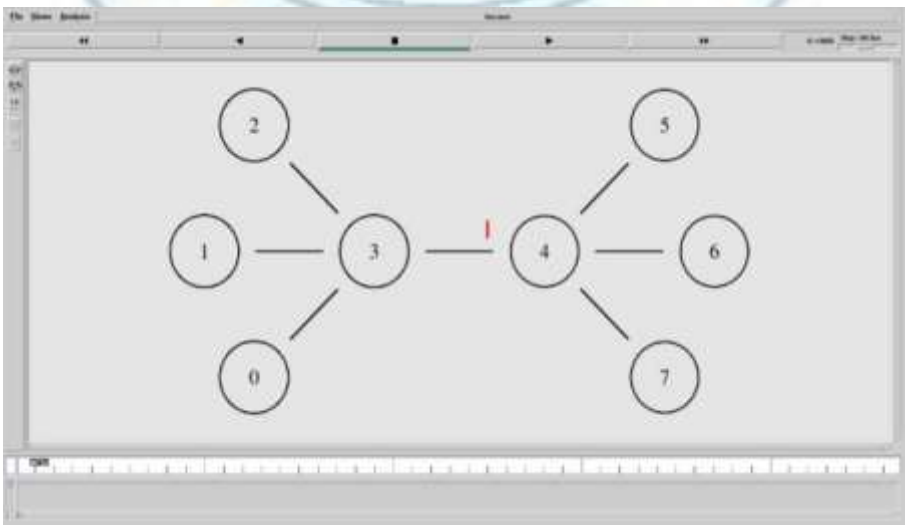
Step 9. Run the Simulation

TCP PROGRAM:

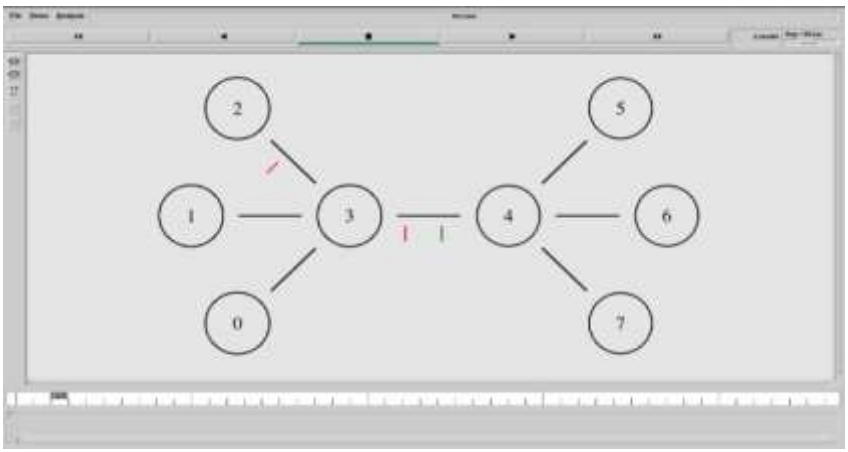
```
set ns [new Simulator]
set nr [open thro_red.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
proc finish {} { global ns nr nf
$ns flush-trace
close $nf close $nr
exec nam thro.nam & exit 0
}
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
$ns duplex-link $n0 $n3 1Mb 10ms RED
$ns duplex-link $n1 $n3 1Mb 10ms RED
$ns duplex-link $n2 $n3 1Mb 10ms RED
$ns duplex-link $n3 $n4 1Mb 10ms RED
$ns duplex-link $n4 $n5 1Mb 10ms RED
$ns duplex-link $n4 $n6 1Mb 10ms RED
$ns duplex-link $n4 $n7 1Mb 10ms RED
$ns duplex-link-op $n0 $n3 orient right-up
```

```
$ns duplex-link-op $n4 $n7 orient right-down
$ns duplex-link-op $n1 $n3 orient right
$ns duplex-link-op $n6 $n4 orient left
set tcp0 [new Agent/TCP]
$ns attach-agent $n2 $tcp0
$ns connect $tcp0 $sink0
set tcp1 [new Agent/TCP]
$ns attach-agent $n0 $tcp2
set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2
set sink2 [new Agent/TCPSink]
$ns attach-agent $n7 $sink2
$ns connect $tcp2 $sink2
$tcp0 set fid_ 1
$tcp1 set fid_ 2
$tcp2 set fid_ 3
$ns color 1 Red
$ns color 2 Green
$ns color 3 Blue
$ns at 0.1 "$ftp0 start"
$ns at 0.2 "$ftp1 start"
$ns at 0.5 "$ftp2 start"
$ns at 4.0 "$ftp2 stop"
$ns at 4.2 "$ftp1 stop"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "finish"
puts "Simulation is starting..."
$ns run
```

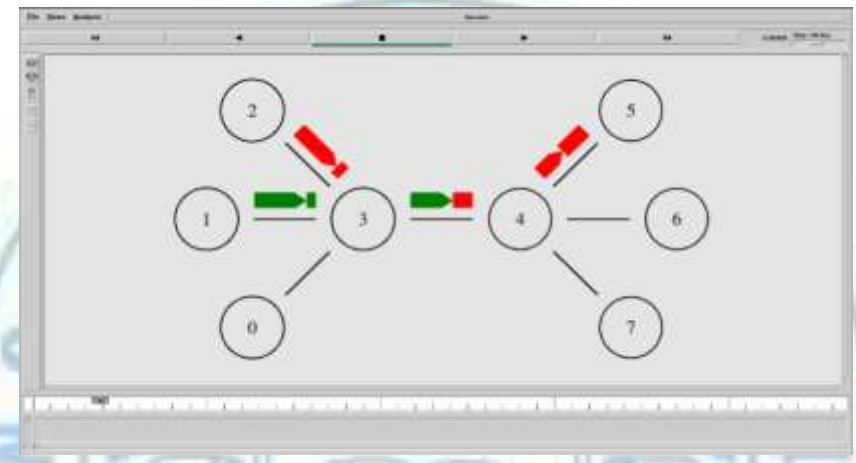
OUTPUT:



Establishment of communication (sender acknowledgement)



Establishment of communication (receiver acknowledgement)



Data transfer

UDP ALGORITHM:

Step 1. Initialize the Simulator

Step 2. Create Nodes

Step 3. Set up Network Links

Step 4. Link Orientations for Visualization

Step 5. Create TCP Traffic Generation Agents

Step 6. Configure Flow Identifiers and Colors

Step 7. Set Traffic Start and Stop Times

Step 8. Finish Procedure

Step 9. Run the Simulation

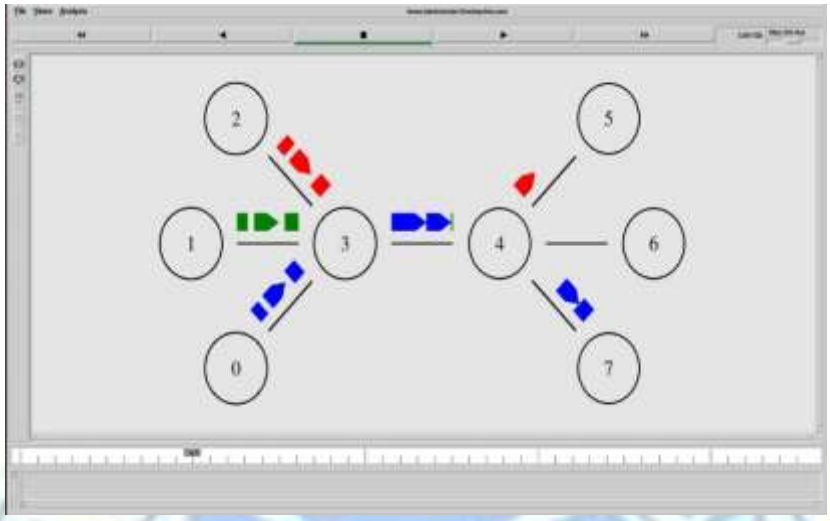
UDP PROGRAM:

```
set ns [new Simulator]
set nr [open thro_red.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
```

```
proc finish {} {
global ns nr nf
$ns flush-trace close $nf
close $nr
exec nam thro.nam & exit 0
}
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
$ns duplex-link $n4 $n5 1Mb 10ms RED
$ns duplex-link $n4 $n6 1Mb 10ms RED
$ns duplex-link $n4 $n7 1Mb 10ms RED
$ns duplex-link-op $n0 $n3 orient right-up
$ns duplex-link-op $n3 $n4 orient middle
$ns duplex-link-op $n4 $n7 orient right-down
$ns duplex-link-op $n1 $n3 orient right
$ns duplex-link-op $n6 $n4 orient left
set udp0 [new Agent/UDP
$ns attach-agent $n2 $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n5 $null0
$ns connect $udp0 $null0
set udp1 [new Agent/UDP]
$ns attach-agent $n1 $udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
set null1 [new Agent/Null]
$ns connect $udp1 $null1
set udp2 [new Agent/UDP]
$ns attach-agent $n0 $udp2
set cbr2 [new Application/Traffic/CBR]
$cbr2 set packetSize_ 500
set null2 [new Agent/Null]
$ns attach-agent $n7 $null2
$ns connect $udp2 $null2
$udp0 set fid_ 1
$udp1 set fid_ 2
$udp2 set fid_ 3
$ns color 1 Red
$ns color 2 Green
$ns color 3 Blue
$ns at 0.1 "$cbr0 start"
$ns at 0.2 "$cbr1 start"
$ns at 0.5 "$cbr2 start"
$ns at 4.0 "$cbr2 stop"
$ns at 4.2 "$cbr1 stop"
```

```
$ns at 4.5 "$cbr0 stop"
$ns at 5.0 "finish"
puts "Simulation is starting..."
$ns run
```

OUTPUT:



RESULT:

Thus, the study of TCP/UDP performance using NS2 is executed and verified successfully.

EX NO: 9A	SIMULATION OF DISTANCE VECTOR ROUTING ALGORITHM
DATE:	

AIM:

To simulate the Distance vector routing protocols using NS2.

ALGORITHM:

Step 1. Create a simulator object

Step 2. Set routing protocol to Distance Vector routing

Step 3. Trace packets on all links onto NAM trace and text trace file

Step 4. Define finish procedure to close files, flush tracing and run NAM

Step 5. Create eight nodes

Step 6. Create CBR traffic on top of UDP and set traffic parameters.

Step 7. Add a sink agent to node n4

Step 8. Connect source and the sink

Step 9. Schedule events as follows:

1. Start traffic flow at 0.5
2. Down the link n3-n4 at 1.0
3. Up the link n3-n4 at 2.0
4. Stop traffic at 3.0
5. Call finish procedure at 5.0

Step 10. Start the scheduler

Step 11. Observe the traffic route when link is up and down

Step 12. View the simulated events and trace file analyze it

Step 13. Stop

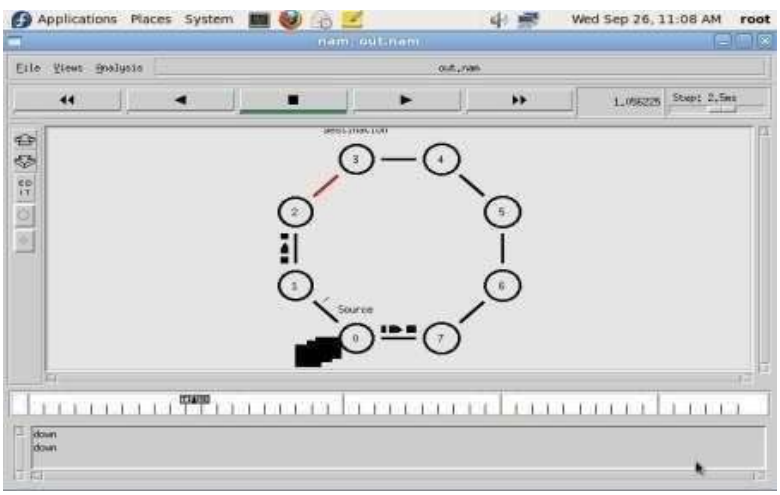
PROGRAM:

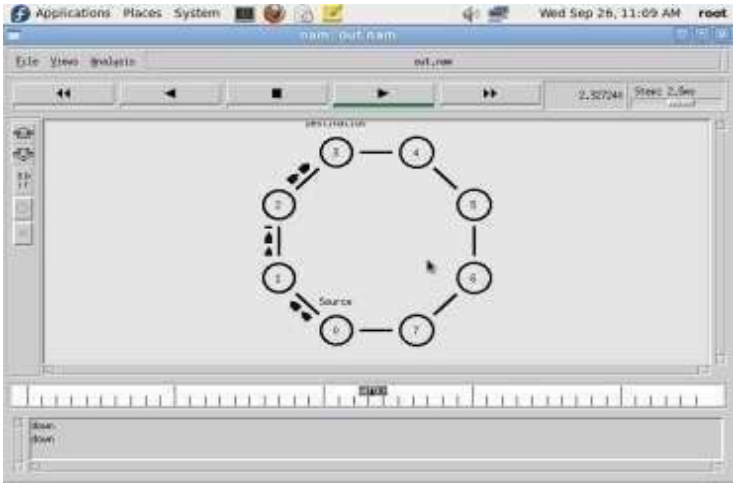
```
set ns [new Simulator]
$ns rtproto DV
set nf [open out.nam w]
$ns namtrace-all $nf
set nt [open trace.tr w]
$ns trace-all $nt
procedure proc finish {}
{
global ns nf
$ns flush-trace
$nf
exec nam -a out.nam &
```

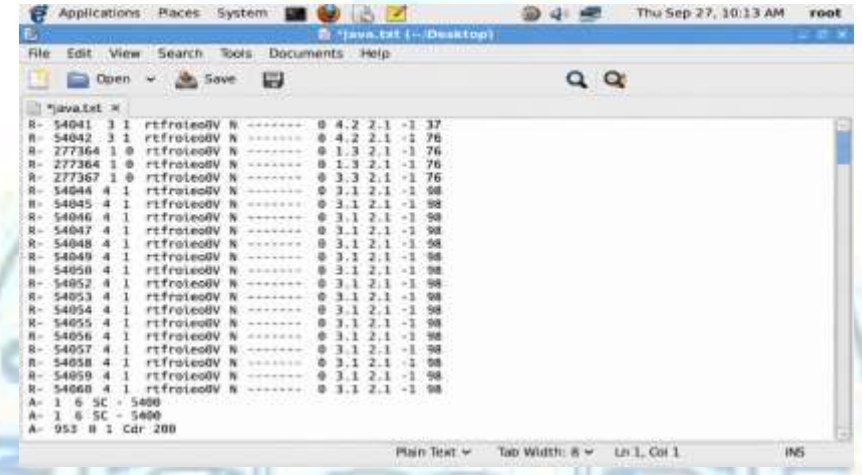
```
exit 0}
set n1 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
$ns duplex-link $n6 $n7 1Mb 10ms DropTail
$ns duplex-link $n7 $n8 1Mb 10ms DropTail
$ns duplex-link $n8 $n1 1Mb 10ms DropTail
$ns duplex-link-op $n2 $n3 orient up
$ns duplex-link-op $n6 $n7 orient down
$ns duplex-link-op $n7 $n8 orient left-down
$ns duplex-link-op $n8 $n1 orient left
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n4 $null0
$ns connect $udp0 $null0
$ns at 0.0 "$n1 label Source"
$ns at 0.0 "$n4 label Destination"
$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n3 $n4
$ns rtmodel-at 2.0 up $n3 $n4
$ns at 4.5 "$cbr0 stop"
$ns at 5.0 "finish"
$ns run
```

OUTPUT:

```
$ ns distvect.tcl
```







RESULT:

Thus, the simulation for Distance vector routing protocols was done using NS2.

EX NO: 9B	SIMULATION OF LINK STATE ROUTING ALGORITHM
DATE:	

AIM:

To simulate the Link state routing protocols using NS2.

ALGORITHM:

Step 1. Create a Simulator object.

Step 2. Set routing as dynamic.

Step 3. Open the trace and nam trace files.

Step 4. Define the finish procedure.

Step 6. Create nodes and the links between them.

Step 7. Create the agents and attach them to the nodes.

Step 8. Create the applications and attach them to the udp agent.

Step 9. Connect udp and null.

Step 10. At 1 sec the link between node 1 and 2 is broken.

Step 11. At 2 sec the link is up again.

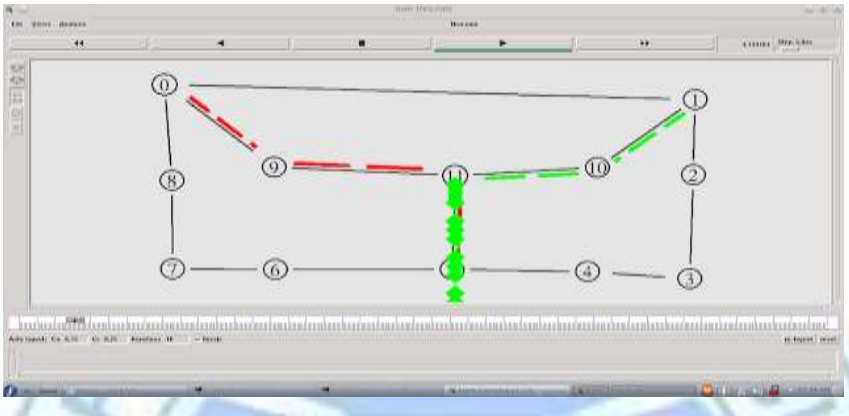
Step 12. Run the simulation.

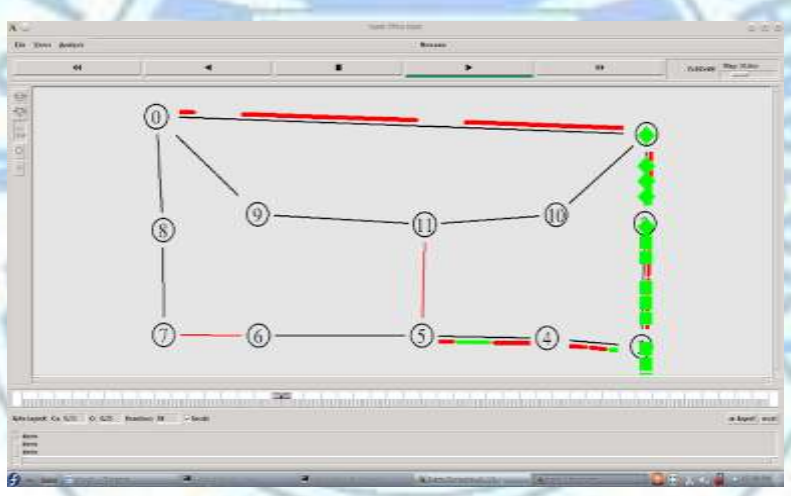
PROGRAM:

```
set ns [new Simulator]
$ns rtproto LS
set nf [open linkstate.nam w]
$ns namtrace-all $nf
set f0 [open linkstate.tr w]
$ns trace-all $f0 proc finish {}
{ global ns f0 nf
$ns flush-trace close $f0 close $nf
exec nam linkstate.nam & exit 0}
for {set i 0} {$i <7} {incr i}
{ set n($i) [$ns node]}
for {set i 0} {$i <7} {incr i} {
$ns duplex-link $n($i) $n([expr ($i+1)%7]) 1Mb 10ms DropTail}
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0 set null0 [new Agent/Null]
$ns attach-agent $n(3) $null0
```

```
$ns connect $udp0 $null0
$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n(1) $n(2)
$ns rtmodel-at 2.0 up $n(1) $n(2)
$ns at 4.5 "$cbr0 stop"
$ns at 5.0 "finish"
$ns run
```

OUTPUT:





RESULT:

Thus, the Link state routing protocols was done using NS2.

EX NO: 10	PERFORMANCE EVALUATION OF ROUTING PROTOCOLS USING SIMULATION TOOL
DATE:	

AIM:

To write a ns2 program for implementing unicast routing protocol.

a) UNICAST ROUTING PROTOCOL ALGORITHM:

Step 1. Start the program.

Step 2. Declare the global variables ns for creating a new simulator.

Step 3. Set the color for packets.

Step 4. Open the network animator file in the name of file2 in the write mode.

Step 5. Open the trace file in the name of file 1 in the write mode.

Step 6. Set the unicast routing protocol to transfer the packets in network.

Step 7. Create the required no of nodes.

Step 8. Create the duplex-link between the nodes including the delay time, bandwidth and dropping queue mechanism.

Step 9. Give the position for the links between the nodes.

Step 10. Set a tcp reno connection for source node.

Step 11. Set the destination node using tcp sink.

Step 12. Setup a ftp connection over the tcp connection.

Step 13. Down the connection between any nodes at a particular time.

Step 14. Reconnect the downed connection at a particular time.

Step 15. Define the finish procedure.

Step 16. In the definition of the finish procedure declare the global variables ns, file1, and file2.

Step 17. Close the trace file and name file and execute the network animation file.

Step 18. At the particular time call the finish procedure.

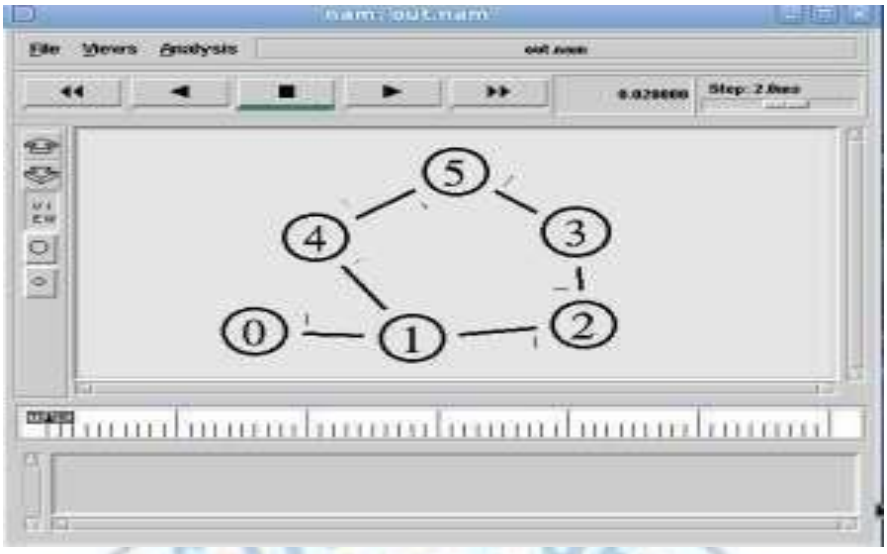
Step 19. Stop the program.

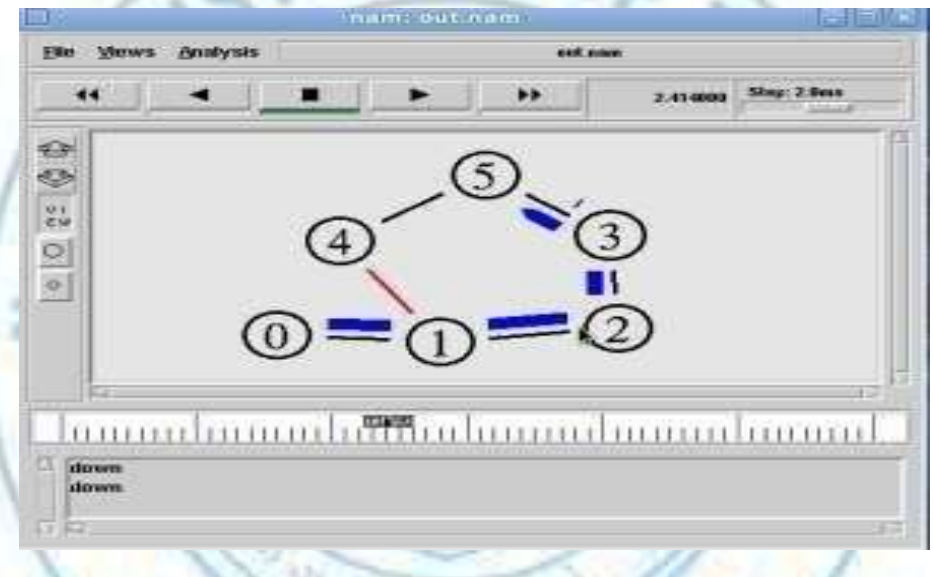
PROGRAM:

```
set ns [new Simulator]
#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
set file1 [open out.tr w]
$ns trace-all $file1
set file2 [open out.nam w]
```

```
{
global ns file1 file2
$ns flush-trace close $file1 close $file2
exec nam out.nam & exit 3
}
$ns rtproto DV
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
$ns duplex-link $n0 $n1 0.3Mb 10ms DropTail
$ns duplex-link $n1 $n2 0.3Mb 10ms DropTail
$ns duplex-link $n2 $n3 0.3Mb 10ms DropTail
$ns duplex-link $n1 $n4 0.3Mb 10ms DropTail
$ns duplex-link $n3 $n5 0.5Mb 10ms DropTail
$ns duplex-link $n4 $n5 0.5Mb 10ms DropTail
$ns duplex-link-op $n0 $n1 orient right
$ns duplex-link-op $n1 $n2 orient right
$ns duplex-link-op $n2 $n3 orient up
$ns duplex-link-op $n1 $n4 orient up-left
$ns duplex-link-op $n3 $n5 orient left-up
$ns duplex-link-op $n4 $n5 orient right-up
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n5 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
$ns rtmodel-at 1.0 down $n1 $n4
$ns rtmodel-at 4.5 up $n1 $n4
$ns at 0.1 "$ftp start"
$ns at 6.0 "finish"
$ns run
```

OUTPUT:





b) MULTICASTING ROUTING PROTOCOL

AIM:

To write a ns2 program for implementing multicasting routing protocol.

ALGORITHM:

Step 1. Start the program.

Step 2. Declare the global variables ns for creating a new simulator.

Step 3. Set the color for packets.

Step 4. Open the network animator file in the name of file2 in the write mode.

Step 5. Open the trace file in the name of file 1 in the write mode.

Step 6. Set the multicast routing protocol to transfer the packets in network.

Step 7. Create the multicast capable no of nodes.

Step 8. Create the duplex-link between the nodes including the delay time, bandwidth and dropping queue mechanism.

Step 9. Give the position for the links between the nodes.

Step 10. Set an udp connection for source node.

Step 11. Set the destination node, port and random false for the source and destination files.

Step 12. Setup a traffic generator CBR for the source and destination files.

Step 13. Down the connection between any nodes at a particular time.

Step 14. Create the receive agent for joining and leaving if the nodes in the group.

Step 15. Define the finish procedure.

Step 16. In the definition of the finish procedure declare the global variables.

Step 17. Close the trace file and namefile and execute the network animation file.

Step 18. At the particular time call the finish procedure.

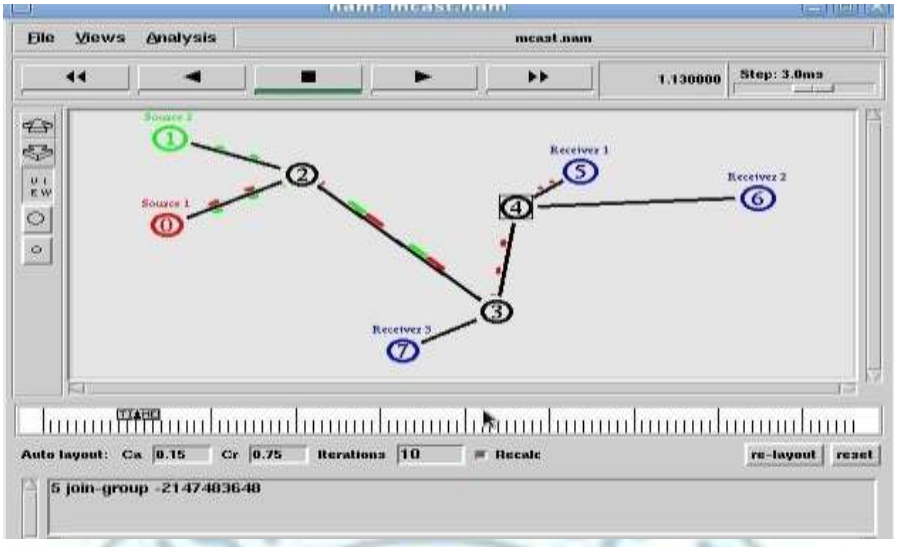
Step 19. Stop the program.

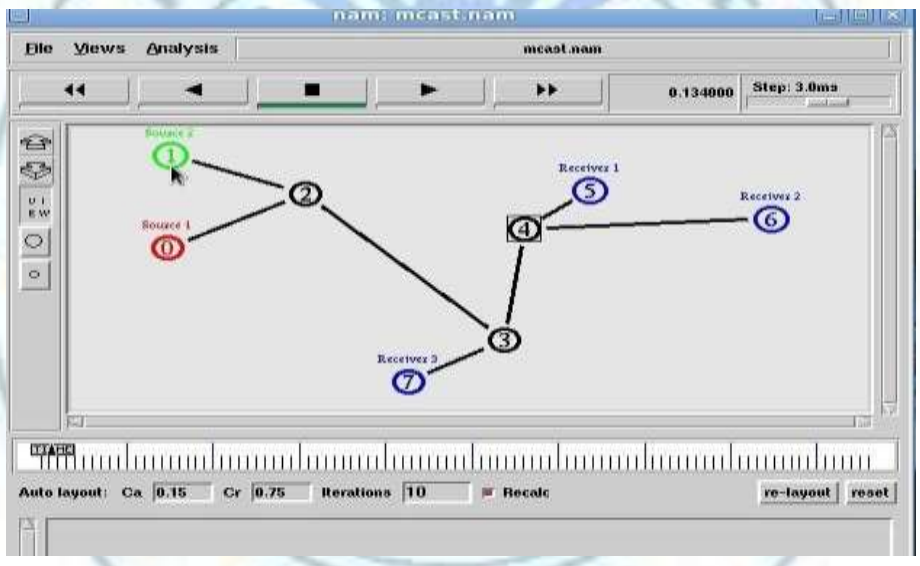
PROGRAM:

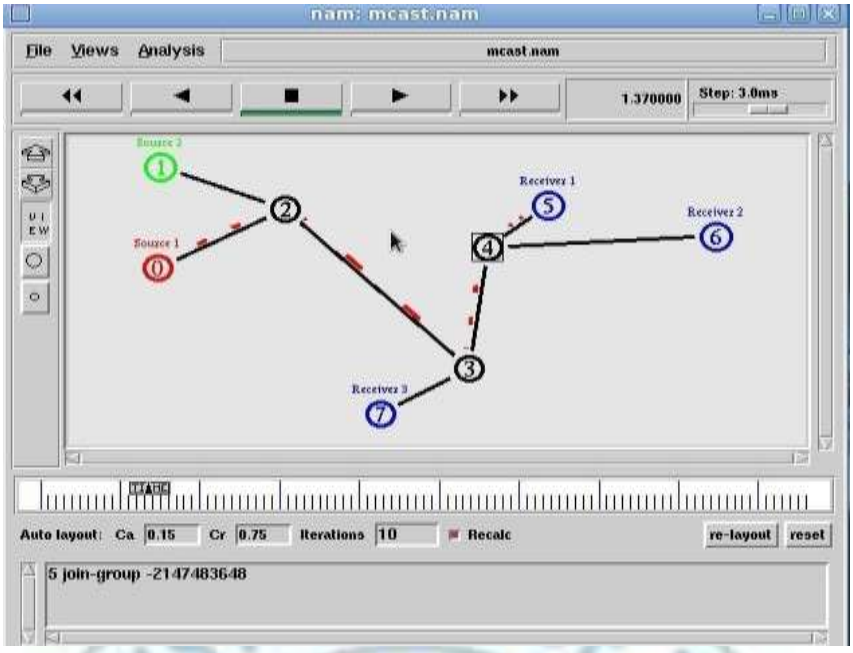
```
set tf [open output.tr w]
$ns trace-all $tf
$ns namtrace-all $fd
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
$ns duplex-link $n0 $n2 1.5Mb 10ms DropTail
$ns duplex-link $n1 $n2 1.5Mb 10ms DropTail
$ns duplex-link $n2 $n3 1.5Mb 10ms DropTail
$ns duplex-link $n3 $n4 1.5Mb 10ms DropTail
$ns duplex-link $n3 $n7 1.5Mb 10ms DropTail
$ns duplex-link $n4 $n5 1.5Mb 10ms DropTail
$ns duplex-link $n4 $n6 1.5Mb 10ms DropTail
mproto DM set mrthandle [$ns mrtproto $mproto {}]
set group1 [Node allocaddr]
set group2 [Node allocaddr]
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0
$udp0 set dst_addr_ $group1
$udp0 set dst_port_ 0
set cbr1 [new Application/Traffic/CBR]
$cbr1 attach-agent $udp0
```

```
$ns attach-agent $n1 $udp1
$udp1 set dst_addr_ $group2
$udp1 set dst_port_ 0
set cbr2 [new Application/Traffic/CBR]
$cbr2 attach-agent $udp1
set rcvr1 [new Agent/Null]
$ns attach-agent $n5 $rcvr1
$ns at 1.0 "$n5 join-group $rcvr1 $group1" set rcvr2 [new Agent/Null]
$ns attach-agent $n6 $rcvr2
$ns at 1.5 "$n6 join-group $rcvr2 $group1" set rcvr3 [new Agent/Null]
$ns attach-agent $n7 $rcvr3
$ns at 3.0 "$n6 join-group $rcvr5 $group2" set rcvr6 [new Agent/Null]
$ns attach-agent $n7 $rcvr3
$ns at 3.5 "$n7 join-group $rcvr6 $group2"
$ns at 4.0 "$n5 leave-group $rcvr1 $group1"
$ns at 4.5 "$n6 leave-group $rcvr2 $group1"
$ns at 5.0 "$n7 leave-group $rcvr3 $group1"
$ns at 5.5 "$n5 leave-group $rcvr4 $group2"
$ns at 6.0 "$n6 leave-group $rcvr5 $group2"
$ns at 6.5 "$n7 leave-group $rcvr6
$group2" # Schedule events
$ns at 0.5 "$cbr1 start"
$ns at 9.5 "$cbr1 stop"
$ns at 0.5 "$cbr2 start"
$ns at 9.5 "$cbr2 stop"
$ns at 10.0 "finish" proc finish {}
{
global ns tf
$ns flush-trace close $tf
exec nam mcast.nam & exit 0
}
$ns color 10 red
$ns color 11 green
$n0 color red
$n0 label "Source 1"
$udp1 set fid_ 11
$n1 color green
$n1 label "Source 2"
$n5 label "Receiver 1"
$n5 color blue
$n6 label "Receiver 2"
$n6 color blue
$n7 label "Receiver 3"
$n7 color blue
$ns set-animation-rate 3.0ms
$ns run
```

OUTPUT:


File Views Analysis
mcast.nam
1.130000
Step: 3.0ms
Source 2
Source 1
Receiver 1
Receiver 2
Receiver 3
Auto layout: Ca 0.15 Cr 0.75 Iterations 10 Recalc
re-layout reset
5 join-group -2147483648


nam: mcast.nam
File Views Analysis
mcast.nam
0.134000
Step: 3.0ms
Source 2
Source 1
Receiver 1
Receiver 2
Receiver 3
Auto layout: Ca 0.15 Cr 0.75 Iterations 10 Recalc
re-layout reset



RESULT:

Thus, the case study about the different routing algorithms to select the network path with its optimum and economical during data transfer is done

.

EX.NO: 11	SIMULATION OF ERROR CORRECTION CODE (LIKE CRC)
DATE:	

AIM:

To implement error checking code using java.

ALGORITHM:

Step 1. Start the Program

Step 2. Given a bit string, append 0S to the end of it (the number of 0s is the same as the degree of the generator polynomial) let B(x) be the polynomial corresponding to B.

Step 3. Divide B(x) by some agreed on polynomial G(x) (generator polynomial) and determine the remainder R(x). This division is to be done using Modulo 2 Division.

Step 4. Define T(x) = B(x) –R(x)

Step 5. (T(x)/G(x) => remainder 0)

Step 6. Transmit T, the bit string corresponding to T(x).

Step 7. Let T‘represent the bit stream the receiver gets and T‘(x) the associated polynomial. The receiver divides T1(x) by G(x).

Step 8. Stop the Program

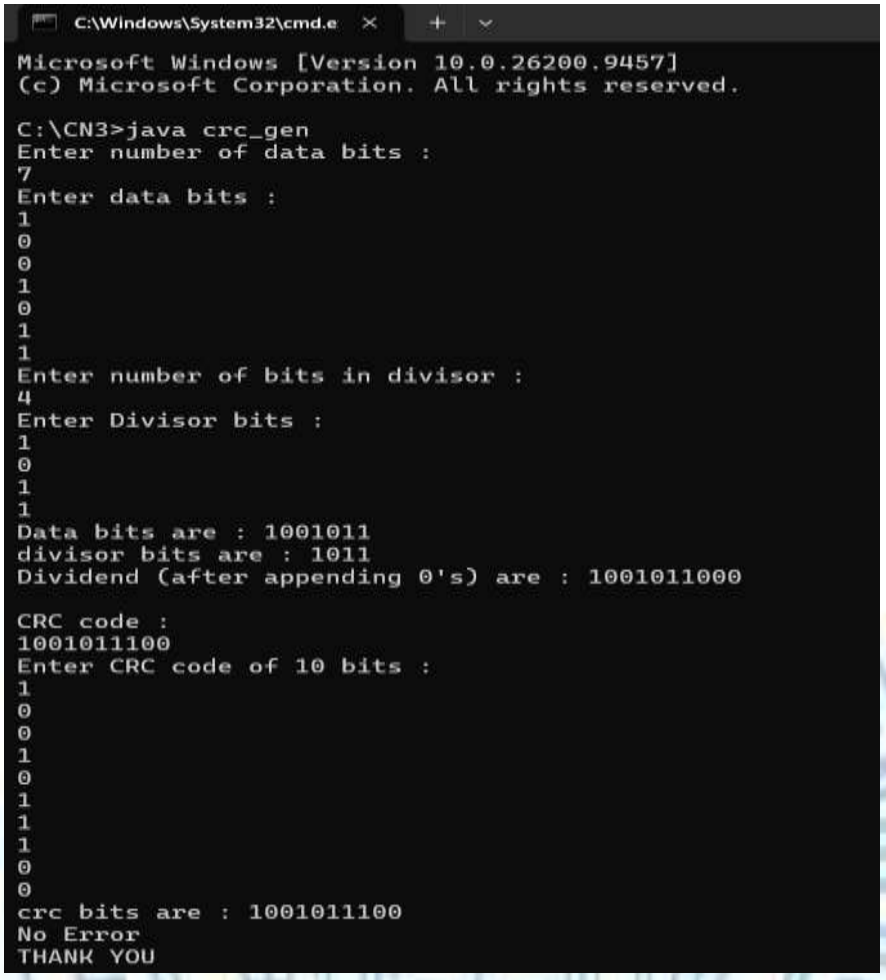
PROGRAM:

```
import java.io.*;
class crc_gen {
public static void main(String args[]) throws IOException {
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int[] data;
int[] div;
int[] dvisor;
int[] rem;
int[] crc;
int data_bits, divisor_bits, tot_length;
System.out.println("Enter number of data bits : ");
data_bits = Integer.parseInt(br.readLine());
data = new int[data_bits];
System.out.println("Enter data bits : ");
for (int i = 0; i < data_bits; i++)
```

```
data[i] = Integer.parseInt(br.readLine());
System.out.println("Enter number of bits in divisor : ");
divisor_bits = Integer.parseInt(br.readLine());
divisor = new int[divisor_bits];
System.out.println("Enter Divisor bits : ");
for (int i = 0; i < divisor_bits; i++)
divisor[i] = Integer.parseInt(br.readLine());
System.out.print("Data bits are : ");
for (int i = 0; i < data_bits; i++)
for (int i = 0; i < divisor_bits; i++)
System.out.print(divisor[i]);
System.out.println();
tot_length = data_bits + divisor_bits - 1;
div = new int[tot_length];
rem = new int[tot_length];
crc = new int[tot_length];
for (int i = 0; i < data.length; i++) div[i] = data[i];
System.out.print("Dividend (after appending 0's) are : ");
for (int i = 0; i < div.length; i++)
System.out.print(div[i]);
System.out.println();
for (int j = 0; j < div.length; j++) {
rem[j] = div[j];
}
rem = divide(div, divisor, rem);
for (int i = 0; i < div.length; i++) {
}
System.out.println("Enter CRC code of " + tot_length + " bits : ");
for (int i = 0; i < crc.length; i++)
crc[i] = Integer.parseInt(br.readLine());
System.out.print("crc bits are : ");
for (int i = 0; i < crc.length; i++)
System.out.print(crc[i]);
System.out.println();
for (int j = 0; j < crc.length; j++) {
```

```
rem[j] = crc[j];
}
rem = divide(crc, divisor, rem);
for (int i = 0; i < rem.length; i++) {
if (rem[i] != 0) {
System.out.println("Error");
break;
}
if (i == rem.length - 1) System.out.println("No Error");
}
System.out.println("THANK YOU)");
}
static int[] divide(int div[], int divisor[], int rem[]) { int cur = 0;
while (true) {
for (int i = 0; i < divisor.length; i++)
rem[cur + i] = (rem[cur + i] ^ divisor[i]);
while (rem[cur] == 0 && cur != rem.length - 1)
cur++;
if ((rem.length - cur) < divisor.length) break;
}
return rem;
}}
```

OUTPUT:



RESULT:

Thus, the above program for error checking code using was executed successfully.